Course 2: Supervised Learning



Course 2: Supervised Learning

Summary

Last session

1. AI definition
2. Applications & Open Issues
3. Deep learning
4. Foundation models

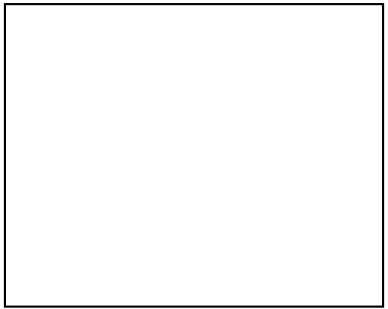
Today's session

- Learning from labeled examples
- Challenges of supervised learning
- Evaluation: ground truth and metrics

Summary
Last session
AI definition
Applications & Open Issues
Deep learning
Foundation models
Today's session
Learning from labeled examples
Challenges of supervised learning
Evaluation: ground truth and metrics

Notations

Vector space ($\mathbb{R}^d$)



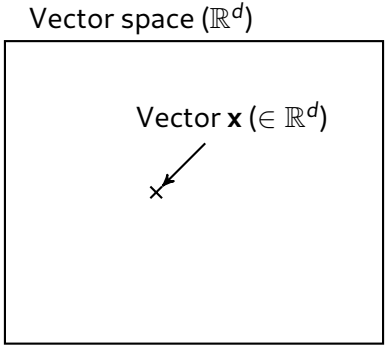
Notations
Vector space ($\mathbb{R}^d$)



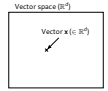
Notations used throughout the course. $\mathbb{R}^d$ is the vector space of real values in dimension d; a single example is a vector $\mathbf{x} \in \mathbb{R}^d$, i.e. a list of d numbers, and X denotes the set of all the examples we have.
The mental image to keep is the one on the slide: **one example = one point in a space**. The rectangle is a 2D drawing of that space, but d is usually much larger (hundreds or thousands), which is exactly what will cause trouble later (see the curse of dimensionality).
Concretely, d is the number of **features** describing one example: for a table, one column per feature; for a 32×32 colour image seen as raw pixels, $d = 32 \times 32 \times 3 = 3072$. Two examples that are similar in the real world should ideally be close in this space; whether they actually are depends entirely on how the features were built, which is the subject of the next slides.

Vocabulary: n (number of examples, the size of X) and d (dimension, the number of features) are the two quantities you should always know about a dataset before doing anything with it.

Notations



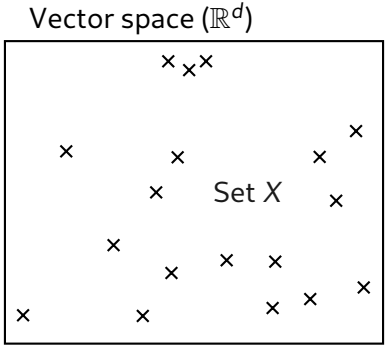
Notations



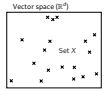
Notations used throughout the course. $\mathbb{R}^d$ is the vector space of real values in dimension d; a single example is a vector $\mathbf{x} \in \mathbb{R}^d$, i.e. a list of d numbers, and X denotes the set of all the examples we have.
The mental image to keep is the one on the slide: **one example = one point in a space**. The rectangle is a 2D drawing of that space, but d is usually much larger (hundreds or thousands), which is exactly what will cause trouble later (see the curse of dimensionality).
Concretely, d is the number of **features** describing one example: for a table, one column per feature; for a 32×32 colour image seen as raw pixels, $d = 32 \times 32 \times 3 = 3072$. Two examples that are similar in the real world should ideally be close in this space; whether they actually are depends entirely on how the features were built, which is the subject of the next slides.

Vocabulary: n (number of examples, the size of X) and d (dimension, the number of features) are the two quantities you should always know about a dataset before doing anything with it.

Notations



Notations



Notations used throughout the course. $\mathbb{R}^d$ is the vector space of real values in dimension d; a single example is a vector $\mathbf{x} \in \mathbb{R}^d$, i.e. a list of d numbers, and X denotes the set of all the examples we have.
The mental image to keep is the one on the slide: **one example = one point in a space**. The rectangle is a 2D drawing of that space, but d is usually much larger (hundreds or thousands), which is exactly what will cause trouble later (see the curse of dimensionality).
Concretely, d is the number of **features** describing one example: for a table, one column per feature; for a 32×32 colour image seen as raw pixels, $d = 32 \times 32 \times 3 = 3072$. Two examples that are similar in the real world should ideally be close in this space; whether they actually are depends entirely on how the features were built, which is the subject of the next slides.

Vocabulary: n (number of examples, the size of X) and d (dimension, the number of features) are the two quantities you should always know about a dataset before doing anything with it.

What is the vector x? (1/2)

Traditional Machine Learning

$\rightarrow$ x is generally a feature representation of the data

Ex: edges in the image, spectrograms in audio, etc.

Deep Learning

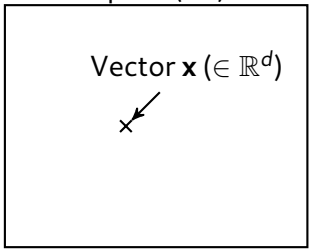
$\rightarrow$ x is the data, or a small transformation of the data

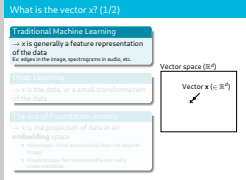
The era of Foundation models

$\rightarrow$ x is the projection of data in an **embedding** space

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable

Vector space ($\mathbb{R}^d$)





In traditional Machine Learning, data is generally a feature representation of the data (i.e. the edges or the colorimetry levels in an image, or spectrograms in audio).

A first shift appears with Deep Learning, where data is fed as it is to models, allowing to alleviate the complex, time-consuming and hand-crafted feature engineering, and leading to impressive performance in image processing, the drawback being a need for large amounts of data and computing power.

Finally, a recent shift appear with larger and more sophisticated models ("Foundation Models"), where they can be used as a first pre-processing step. The resulting representations are called "embeddings", and are vectors in the latent space of the model, i.e. the input vectors (image, text, ...) that have been mapped in a lower dimensional space by a function $f : R^d \rightarrow R^l$. Embeddings are usually richer semantically and easier to manipulate for a downstream task (e.g. classification).

What is the vector x? (1/2)

Traditional Machine Learning

$\rightarrow$ x is generally a feature representation of the data
Ex: edges in the image, spectrograms in audio, etc.

Deep Learning

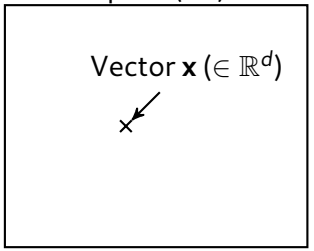
$\rightarrow$ x is the data, or a small transformation of the data

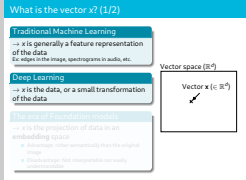
The era of Foundation models

$\rightarrow$ x is the projection of data in an **embedding** space

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable

Vector space ($\mathbb{R}^d$)





In traditional Machine Learning, data is generally a feature representation of the data (i.e. the edges or the colorimetry levels in an image, or spectrograms in audio).
A first shift appears with Deep Learning, where data is fed as it is to models, allowing to alleviate the complex, time-consuming and hand-crafted feature engineering, and leading to impressive performance in image processing, the drawback being a need for large amounts of data and computing power.
Finally, a recent shift appear with larger and more sophisticated models ("Foundation Models"), where they can be used as a first pre-processing step. The resulting representations are called "embeddings", and are vectors in the latent space of the model, i.e. the input vectors (image, text, ...) that have been mapped in a lower dimensional space by a function $f : R^d \rightarrow R^l$. Embeddings are usually richer semantically and easier to manipulate for a downstream task (e.g. classification).

What is the vector x? (1/2)

Traditional Machine Learning

$\rightarrow$ x is generally a feature representation of the data

Ex: edges in the image, spectrograms in audio, etc.

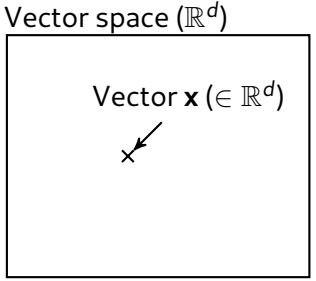
Deep Learning

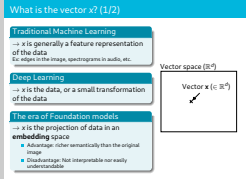
$\rightarrow$ x is the data, or a small transformation of the data

The era of Foundation models

$\rightarrow$ x is the projection of data in an **embedding** space

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable



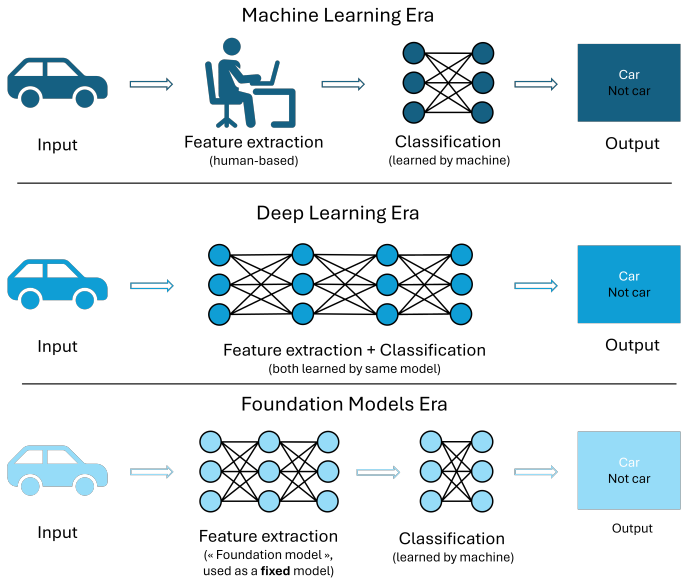


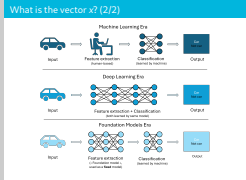
In traditional Machine Learning, data is generally a feature representation of the data (i.e. the edges or the colorimetry levels in an image, or spectrograms in audio).

A first shift appears with Deep Learning, where data is fed as it is to models, allowing to alleviate the complex, time-consuming and hand-crafted feature engineering, and leading to impressive performance in image processing, the drawback being a need for large amounts of data and computing power.

Finally, a recent shift appear with larger and more sophisticated models ("Foundation Models"), where they can be used as a first pre-processing step. The resulting representations are called "embeddings", and are vectors in the latent space of the model, i.e. the input vectors (image, text, ...) that have been mapped in a lower dimensional space by a function $f : R^d \rightarrow R^l$. Embeddings are usually richer semantically and easier to manipulate for a downstream task (e.g. classification).

What is the vector x? (2/2)





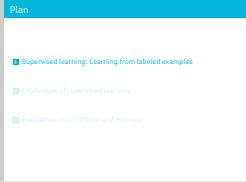
A schematic representation of the three types of input vectors x seen in the previous slide: handcrafted features, raw data, and embeddings.

Plan

1 Supervised learning: Learning from labeled examples

2 Challenges of supervised learning

3 Evaluation: ground truth and metrics



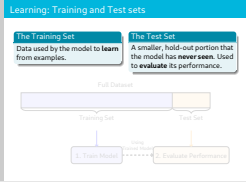
Learning: Training and Test sets

The Training Set

Data used by the model to **learn** from examples.

The Test Set

A smaller, hold-out portion that the model has **never seen**. Used to **evaluate** its performance.



Rule number one: **never evaluate a model on the data it was trained on**. Reproducing answers already seen is memorisation, and a lookup table does it perfectly. What we want is **generalisation**: being right on examples unseen during training, since those are the ones the model will meet once deployed. The test set simulates that situation.

In practice: splits of 80% training/20%test, or 70/30, and the split must be **random** (otherwise, you may have representation problems: if your file is sorted by class, the last 20% may contain a single class). In `sklearn`, `train_test_split` does it, and fixing `random_state` makes the split reproducible, which is needed to compare models fairly.

The cardinal sin is **data leakage**: any information from the test set reaching the training procedure (e.g., duplicated examples): always be careful to avoid it, and check your data for duplicates before splitting.

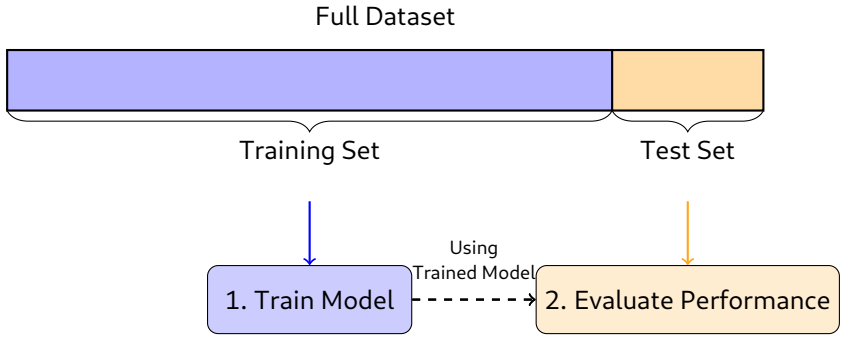
Learning: Training and Test sets

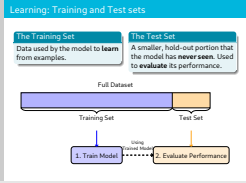
The Training Set

Data used by the model to **learn** from examples.

The Test Set

A smaller, hold-out portion that the model has **never seen**. Used to **evaluate** its performance.





Rule number one: **never evaluate a model on the data it was trained on**. Reproducing answers already seen is memorisation, and a lookup table does it perfectly. What we want is **generalisation**: being right on examples unseen during training, since those are the ones the model will meet once deployed. The test set simulates that situation.

In practice: splits of 80% training/20%test, or 70/30, and the split must be **random** (otherwise, you may have representation problems: if your file is sorted by class, the last 20% may contain a single class). In `sklearn`, `train_test_split` does it, and fixing `random_state` makes the split reproducible, which is needed to compare models fairly.

The cardinal sin is **data leakage**: any information from the test set reaching the training procedure (e.g., duplicated examples): always be careful to avoid it, and check your data for duplicates before splitting.

Supervised learning

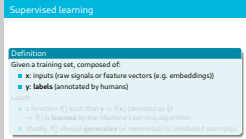
Definition

Given a training set, composed of:

- **x**: inputs (raw signals or feature vectors (e.g. embeddings))
- **y**: **labels** (annotated by humans)

Learn:

- a function $f()$ such that $\mathbf{y} \approx f(\mathbf{x})$ (denoted as $\hat{\mathbf{y}}$)
  $\Rightarrow f()$ is **learned** by the Machine Learning algorithm
- Ideally, $f()$ should **generalize** ($\neq$ memorize) to unlabeled examples.



In supervised learning, we are given **pairs** $(\mathbf{x}, \mathbf{y})$: an input and the answer expected for it. In this context, someone (typically a human expert) has annotated the data with the correct answers.
The goal is to find a function f such that $f(\mathbf{x}) \approx \mathbf{y}$ on the training pairs. Note the "$\approx$": we are **not** looking for a perfect equality in most cases, because that generally lead to overfitting (see later): a function that fit so much the training answers that it also fit irrelvant components, leading in turn to poor generalization on examples that were never annotated.
A remark on algorithms: f is chosen inside a restricted family of functions (linear functions, decision trees, neural networks…), fixed by the algorithm you pick; "learning" means adapting one member of that family to your data, by adjusting its parameters.

Producing the (human annotated) labels **y** is usually the bottleneck of a real project: it is slow, expensive, requires expertise, and its quality directly caps the quality of the model.

Supervised learning

Definition

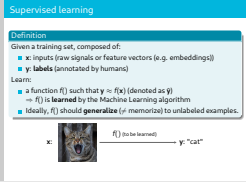
Given a training set, composed of:

- **x**: inputs (raw signals or feature vectors (e.g. embeddings))
- **y**: **labels** (annotated by humans)

Learn:

- a function $f()$ such that $\mathbf{y} \approx f(\mathbf{x})$ (denoted as $\hat{\mathbf{y}}$)
  $\Rightarrow$ $f()$ is **learned** by the Machine Learning algorithm
- Ideally, $f()$ should **generalize** ($\neq$ memorize) to unlabeled examples.

x:  $f()$ (to be learned) → **y**: "cat"



In supervised learning, we are given **pairs** $(\mathbf{x}, \mathbf{y})$: an input and the answer expected for it. In this context, someone (typically a human expert) has annotated the data with the correct answers.

The goal is to find a function f such that $f(\mathbf{x}) \approx \mathbf{y}$ on the training pairs. Note the "$\approx$": we are **not** looking for a perfect equality in most cases, because that generally lead to overfitting (see later): a function that fit so much the training answers that it also fit irrelvant components, leading in turn to poor generalization on examples that were never annotated.

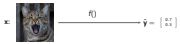
A remark on algorithms: f is chosen inside a restricted family of functions (linear functions, decision trees, neural networks…), fixed by the algorithm you pick; "learning" means adapting one member of that family to your data, by adjusting its parameters.

Producing the (human annotated) labels **y** is usually the bottleneck of a real project: it is slow, expensive, requires expertise, and its quality directly caps the quality of the model.

Supervised learning: in practice

$\mathbf{x}$:  $\xrightarrow{\quad f() \quad}$ $\hat{\mathbf{y}} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix}$

Supervised learning: in practice



Some important details:

Encoding. A class name is not a number, and numbering classes $1, 2, 3$ would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

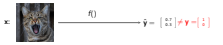
Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification.

Training. Summed over all examples, the loss is a single number depending on the parameters of f; learning is minimising it, typically by gradient descent (Session 1, and, in near future, Session 4). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**

Supervised learning: in practice

$\mathbf{x}$:  $\xrightarrow{f()}$ $\hat{\mathbf{y}} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix} \neq \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

Supervised learning: in practice



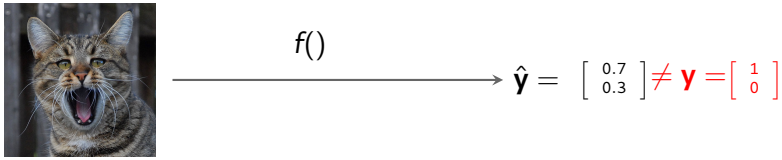
Some important details:

Encoding. A class name is not a number, and numbering classes $1, 2, 3$ would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification.

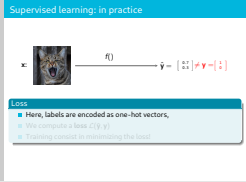
Training. Summed over all examples, the loss is a single number depending on the parameters of f; learning is minimising it, typically by gradient descent (Session 1, and, in near future, Session 4). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**

Supervised learning: in practice



Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
- Training consist in minimizing the loss!



Some important details:

Encoding. A class name is not a number, and numbering classes 1, 2, 3 would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification.

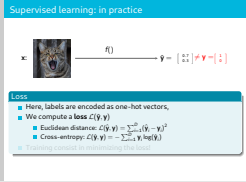
Training. Summed over all examples, the loss is a single number depending on the parameters of f; learning is minimising it, typically by gradient descent (Session 1, and, in near future, Session 4). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**

Supervised learning: in practice

$\mathbf{x}$:  $\xrightarrow{f()}$ $\hat{\mathbf{y}} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix} \neq \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
  - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^{D} (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
  - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{i=1}^{D} \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!



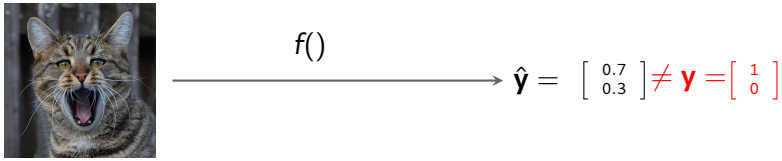
Some important details:

Encoding. A class name is not a number, and numbering classes 1, 2, 3 would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification.

Training. Summed over all examples, the loss is a single number depending on the parameters of f; learning is minimising it, typically by gradient descent (Session 1, and, in near future, Session 4). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**

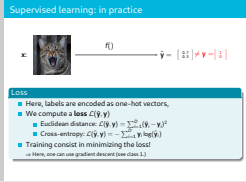
Supervised learning: in practice



Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
  - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^{D} (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
  - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{i=1}^{D} \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!

  $\Rightarrow$ Here, one can use gradient descent (see class 1.)



Some important details:

Encoding. A class name is not a number, and numbering classes 1, 2, 3 would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification.

Training. Summed over all examples, the loss is a single number depending on the parameters of f; learning is minimising it, typically by gradient descent (Session 1, and, in near future, Session 4). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**

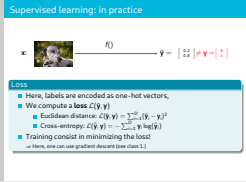
Supervised learning: in practice

$\mathbf{x}$:  $\xrightarrow{f()}$ $\hat{\mathbf{y}} = \begin{bmatrix} 0.2 \\ 0.8 \end{bmatrix} \neq \mathbf{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
  - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^{D} (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
  - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{i=1}^{D} \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!

  $\Rightarrow$ Here, one can use gradient descent (see class 1.)



Some important details:

Encoding. A class name is not a number, and numbering classes 1, 2, 3 would wrongly suggest that class 1 is closer to 2 than to 3. Categorical labels are therefore **one-hot** encoded: a vector of length D full of zeros with a single 1 at the correct class. The model outputs D scores, and the predicted class is the position of the maximum.

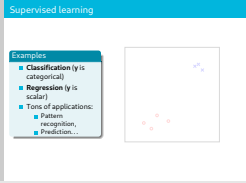
Measuring the error. Both being vectors, their disagreement is measured by a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, large when they differ, zero when they coincide. The squared Euclidean distance is natural for regression, cross-entropy is standard for classification.

Training. Summed over all examples, the loss is a single number depending on the parameters of f; learning is minimising it, typically by gradient descent (Session 1, and, in near future, Session 4). Careful: the loss is minimised on the *training* set, while what matters is the error on the *test* set. Once again: **Never optimise the model on the test set!**

Supervised learning

Examples

- **Classification** (**y** is categorical)
- **Regression** (**y** is scalar)
- Tons of applications:
  - Pattern recognition,
  - Prediction…



Two families, depending on the **nature of y**

- **Classification**: **y** is categorical, taking values in a finite unordered set (apple vs. orange, spam vs. legitimate, one of ten digits).
- **Regression**: **y** is a continuous scalar (a price, tomorrow's temperature).

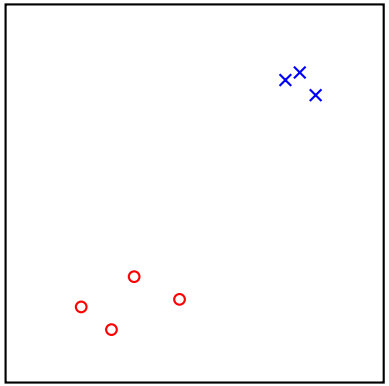
The problem's family determines the algorithms, the losses and the metrics. The metrics at the end of this course (accuracy, precision, recall) are specific to classification; regression uses the mean squared or mean absolute error.

On the figure, red circles and blue crosses are the training examples, drawn as points in the 2D space of the first slide. Learning a classifier amounts to **cutting the plane into regions**, one predicted red and the other blue; a new point is classified by the region it falls into. Once again, the question is not whether the boundary separates *these* points (many do) but whether it is still right on points we have not seen.

Supervised learning

Examples

- **Classification** (**y** is categorical)
- **Regression** (**y** is scalar)
- Tons of applications:
  - Pattern recognition,
  - Prediction…



Supervised learning
Examples
Classification (y is categorical)
Regression (y is scalar)
Tons of applications:
Pattern recognition,
Prediction…

Two families, depending on the **nature of y**

- **Classification**: **y** is categorical, taking values in a finite unordered set (apple vs. orange, spam vs. legitimate, one of ten digits).
- **Regression**: **y** is a continuous scalar (a price, tomorrow's temperature).

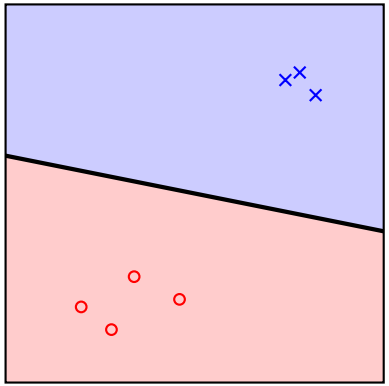
The problem's family determines the algorithms, the losses and the metrics. The metrics at the end of this course (accuracy, precision, recall) are specific to classification; regression uses the mean squared or mean absolute error.

On the figure, red circles and blue crosses are the training examples, drawn as points in the 2D space of the first slide. Learning a classifier amounts to **cutting the plane into regions**, one predicted red and the other blue; a new point is classified by the region it falls into. Once again, the question is not whether the boundary separates *these* points (many do) but whether it is still right on points we have not seen.

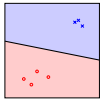
Supervised learning

Examples

- **Classification** (**y** is categorical)
- **Regression** (**y** is scalar)
- Tons of applications:
  - Pattern recognition,
  - Prediction…



Supervised learning
Examples
Classification (y is categorical)
Regression (y is scalar)
Tons of applications:
Pattern recognition,
Prediction…



Two families, depending on the **nature of y**

- **Classification**: **y** is categorical, taking values in a finite unordered set (apple vs. orange, spam vs. legitimate, one of ten digits).
- **Regression**: **y** is a continuous scalar (a price, tomorrow's temperature).

The problem's family determines the algorithms, the losses and the metrics. The metrics at the end of this course (accuracy, precision, recall) are specific to classification; regression uses the mean squared or mean absolute error.

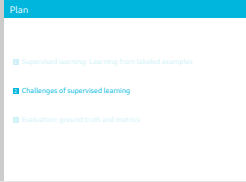
On the figure, red circles and blue crosses are the training examples, drawn as points in the 2D space of the first slide. Learning a classifier amounts to **cutting the plane into regions**, one predicted red and the other blue; a new point is classified by the region it falls into. Once again, the question is not whether the boundary separates *these* points (many do) but whether it is still right on points we have not seen.

Plan

1 Supervised learning: Learning from labeled examples

2 Challenges of supervised learning

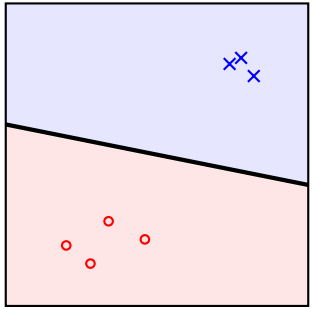
3 Evaluation: ground truth and metrics



Infinite solutions and priors

An ill-defined problem

- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.



Infinite solutions and priors
An ill-defined problem
An infinity of potential solutions, one must be the “best one” but is unreachable,
⇒ requires priors or constraints.
Training on more data is a way to reduce the number of potential solutions, but it is not always available.



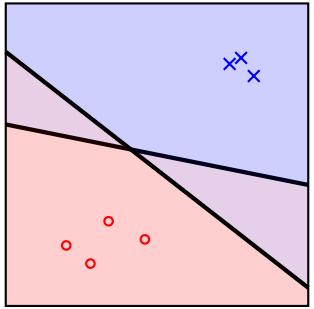
In general, the supervised problem is ill-posed, because many solutions are consistent with the training data. Here for instance„ the three boundaries classify the training points equally well, and nothing in the data lets us prefer one. Yet they disagree on new points, as the “?” shows: depending on the boundary, that point is red or blue. A finite set of examples is consistent with an infinity of functions, and the best one (which would be right on all future inputs) is unreachable by construction (we don’t have the future examples!).
Two ways out:

1. **Add priors or constraints**: restrict the search to functions we consider reasonable. Choosing an algorithm does exactly this, thus comparing algorithms is informative: each embodies a different prior.
2. **Add data**: every new labeled example eliminates the functions that contradict it.

Infinite solutions and priors

An ill-defined problem

- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.



Infinite solutions and priors
An ill-defined problem
An infinity of potential solutions, one must be the “best one” but is unreachable,
⇒ requires priors or constraints.
Training on more data is a way to reduce the number of potential solutions, but it is not always available.



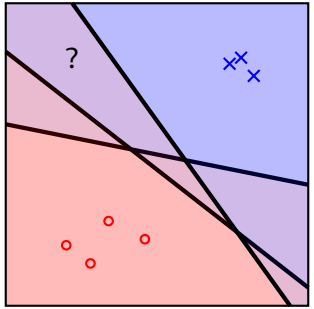
In general, the supervised problem is ill-posed, because many solutions are consistent with the training data. Here for instance„ the three boundaries classify the training points equally well, and nothing in the data lets us prefer one. Yet they disagree on new points, as the “?” shows: depending on the boundary, that point is red or blue. A finite set of examples is consistent with an infinity of functions, and the best one (which would be right on all future inputs) is unreachable by construction (we don’t have the future examples!).
Two ways out:

1. **Add priors or constraints**: restrict the search to functions we consider reasonable. Choosing an algorithm does exactly this, thus comparing algorithms is informative: each embodies a different prior.
2. **Add data**: every new labeled example eliminates the functions that contradict it.

Infinite solutions and priors

An ill-defined problem

- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.



Infinite solutions and priors
An ill-defined problem
An infinity of potential solutions, one must be the “best one” but is unreachable,
⇒ requires priors or constraints.
Training on more data is a way to reduce the number of potential solutions, but it is not always available.
?

In general, the supervised problem is ill-posed, because many solutions are consistent with the training data. Here for instance„ the three boundaries classify the training points equally well, and nothing in the data lets us prefer one. Yet they disagree on new points, as the “?” shows: depending on the boundary, that point is red or blue. A finite set of examples is consistent with an infinity of functions, and the best one (which would be right on all future inputs) is unreachable by construction (we don’t have the future examples!).
Two ways out:

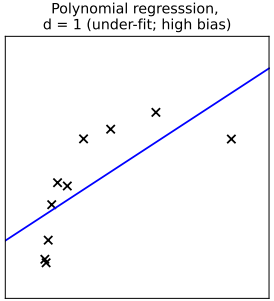
1. **Add priors or constraints**: restrict the search to functions we consider reasonable. Choosing an algorithm does exactly this, thus comparing algorithms is informative: each embodies a different prior.
2. **Add data**: every new labeled example eliminates the functions that contradict it.

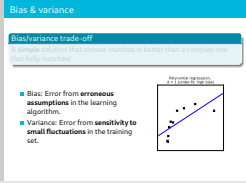
Bias & variance

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.





The same points fitted by a polynomial of degree $d = 1$, 2 and 6.
$d = 1$: a line cannot follow the data. Too rigid, wrong everywhere, and more points would not help. This is **high bias**: the error comes from an assumption built into the model itself.
$d = 6$: the curve passes through every point... but is the worst of the three. Between the points it oscillates, so predictions elsewhere are unreliable, and moving a single training point would change it completely. This is **high variance**: the model absorbed the particularities, and the noise, of this training set.
The trade-off: more capacity lowers the bias and raises the variance, so the total error is minimal at intermediate complexity. Hence the message of the slide. The same phenomenon appears everywhere with a different knob: degree of a polynomial, depth of a tree, number of neighbours k in a KNN (small k = high variance), size of a network.

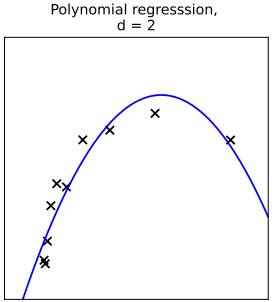
(Rapid) Diagnostic: bad on *both* train and test = bias, model too simple; excellent on train and poor on test = variance, model too complex.

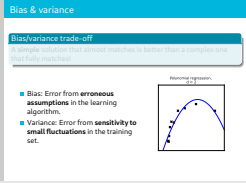
Bias & variance

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.





The same points fitted by a polynomial of degree $d = 1$, 2 and 6.
$d = 1$: a line cannot follow the data. Too rigid, wrong everywhere, and more points would not help. This is **high bias**: the error comes from an assumption built into the model itself.
$d = 6$: the curve passes through every point... but is the worst of the three. Between the points it oscillates, so predictions elsewhere are unreliable, and moving a single training point would change it completely. This is **high variance**: the model absorbed the particularities, and the noise, of this training set.
The trade-off: more capacity lowers the bias and raises the variance, so the total error is minimal at intermediate complexity. Hence the message of the slide. The same phenomenon appears everywhere with a different knob: degree of a polynomial, depth of a tree, number of neighbours k in a KNN (small k = high variance), size of a network.

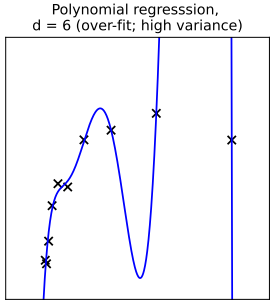
(Rapid) Diagnostic: bad on *both* train and test = bias, model too simple; excellent on train and poor on test = variance, model too complex.

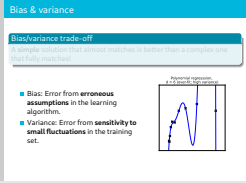
Bias & variance

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.





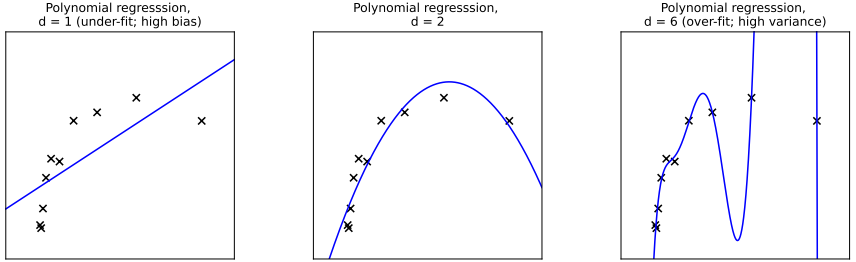
The same points fitted by a polynomial of degree $d = 1$, 2 and 6.
$d = 1$: a line cannot follow the data. Too rigid, wrong everywhere, and more points would not help. This is **high bias**: the error comes from an assumption built into the model itself.
$d = 6$: the curve passes through every point... but is the worst of the three. Between the points it oscillates, so predictions elsewhere are unreliable, and moving a single training point would change it completely. This is **high variance**: the model absorbed the particularities, and the noise, of this training set.
The trade-off: more capacity lowers the bias and raises the variance, so the total error is minimal at intermediate complexity. Hence the message of the slide. The same phenomenon appears everywhere with a different knob: degree of a polynomial, depth of a tree, number of neighbours k in a KNN (small k = high variance), size of a network.

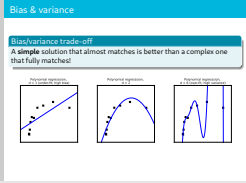
(Rapid) Diagnostic: bad on *both* train and test = bias, model too simple; excellent on train and poor on test = variance, model too complex.

Bias & variance

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!





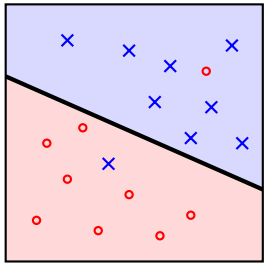
The same points fitted by a polynomial of degree $d = 1$, 2 and 6.
$d = 1$: a line cannot follow the data. Too rigid, wrong everywhere, and more points would not help. This is **high bias**: the error comes from an assumption built into the model itself.
$d = 6$: the curve passes through every point... but is the worst of the three. Between the points it oscillates, so predictions elsewhere are unreliable, and moving a single training point would change it completely. This is **high variance**: the model absorbed the particularities, and the noise, of this training set.
The trade-off: more capacity lowers the bias and raises the variance, so the total error is minimal at intermediate complexity. Hence the message of the slide. The same phenomenon appears everywhere with a different knob: degree of a polynomial, depth of a tree, number of neighbours k in a KNN (small k = high variance), size of a network.

(Rapid) Diagnostic: bad on *both* train and test = bias, model too simple; excellent on train and poor on test = variance, model too complex.

Overfitting

Overfitting

Mimicking is not learning: a model can be **perfect on the training set** and **bad on new data**.



Simple model: **89%** of training points

Overfitting
Overfitting
Mimicking is not learning: a model can be **perfect on the training set** and **bad on new data.**
Simple model: **89%** of training points

Let's assume a toy dataset with two "noisy" points (on the picture on the right: a red circle among the blue crosses, and the opposite): mislabeled or atypical examples, which exist in any real dataset. The straight boundary gets 16 of the 18 training points right (89%). The complex one gets 100%, by wiggling and creating two small "islands" around the noisy points. On five *new* points (yellow halo), the simple boundary is always right, the complex one only twice: the islands and wiggles were built on noise. This is **overfitting**: learning the particularities of the training set instead of the general rule.

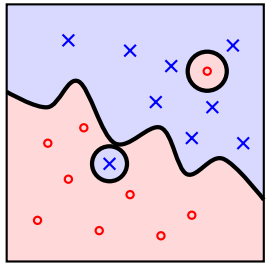
We then displays (on the right) the same phenomenon, but with error curves. In figure, while the training error keeps decreasing towards zero, the error on unseen data is a parabola. It first decreases, then increases again once the model starts learning the noise. The best model is at the minimum of the orange curve; a growing gap between the two curves is the symptom (e.g. 100% on train and 65% on test).

Drawing the orange curve requires unseen data: this is the subject of the next slide (validation set).

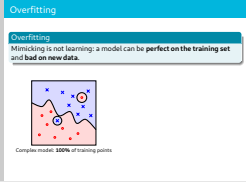
Overfitting

Overfitting

Mimicking is not learning: a model can be **perfect on the training set** and **bad on new data**.



Complex model: **100%** of training points



Let's assume a toy dataset with two "noisy" points (on the picture on the right: a red circle among the blue crosses, and the opposite): mislabeled or atypical examples, which exist in any real dataset. The straight boundary gets 16 of the 18 training points right (89%). The complex one gets 100%, by wiggling and creating two small "islands" around the noisy points. On five *new* points (yellow halo), the simple boundary is always right, the complex one only twice: the islands and wiggles were built on noise. This is **overfitting**: learning the particularities of the training set instead of the general rule.

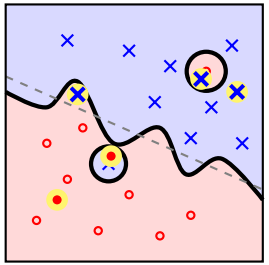
We then displays (on the right) the same phenomenon, but with error curves. In figure, while the training error keeps decreasing towards zero, the error on unseen data is a parabola. It first decreases, then increases again once the model starts learning the noise. The best model is at the minimum of the orange curve; a growing gap between the two curves is the symptom (e.g. 100% on train and 65% on test).

Drawing the orange curve requires unseen data: this is the subject of the next slide (validation set).

Overfitting

Overfitting

Mimicking is not learning: a model can be **perfect on the training set** and **bad on new data**.



New points: simple **5/5**, complex **2/5**



Let's assume a toy dataset with two "noisy" points (on the picture on the right: a red circle among the blue crosses, and the opposite): mislabeled or atypical examples, which exist in any real dataset. The straight boundary gets 16 of the 18 training points right (89%). The complex one gets 100%, by wiggling and creating two small "islands" around the noisy points. On five *new* points (yellow halo), the simple boundary is always right, the complex one only twice: the islands and wiggles were built on noise. This is **overfitting**: learning the particularities of the training set instead of the general rule.

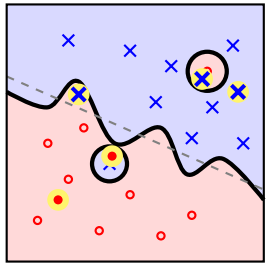
We then displays (on the right) the same phenomenon, but with error curves. In figure, while the training error keeps decreasing towards zero, the error on unseen data is a parabola. It first decreases, then increases again once the model starts learning the noise. The best model is at the minimum of the orange curve; a growing gap between the two curves is the symptom (e.g. 100% on train and 65% on test).

Drawing the orange curve requires unseen data: this is the subject of the next slide (validation set).

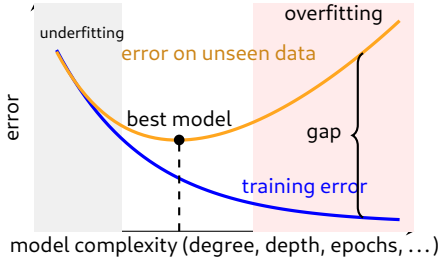
Overfitting

Overfitting

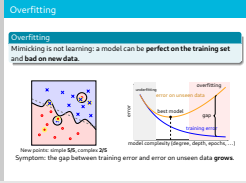
Mimicking is not learning: a model can be **perfect on the training set** and **bad on new data**.



New points: simple **5/5**, complex **2/5**



Symptom: the gap between training error and error on unseen data **grows**.



Let's assume a toy dataset with two "noisy" points (on the picture on the right: a red circle among the blue crosses, and the opposite): mislabeled or atypical examples, which exist in any real dataset. The straight boundary gets 16 of the 18 training points right (89%). The complex one gets 100%, by wiggling and creating two small "islands" around the noisy points. On five *new* points (yellow halo), the simple boundary is always right, the complex one only twice: the islands and wiggles were built on noise. This is **overfitting**: learning the particularities of the training set instead of the general rule.

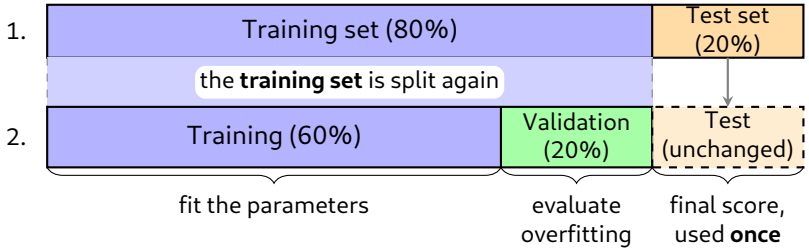
We then displays (on the right) the same phenomenon, but with error curves. In figure, while the training error keeps decreasing towards zero, the error on unseen data is a parabola. It first decreases, then increases again once the model starts learning the noise. The best model is at the minimum of the orange curve; a growing gap between the two curves is the symptom (e.g. 100% on train and 65% on test).

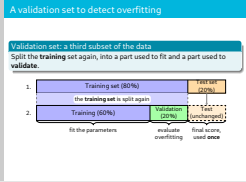
Drawing the orange curve requires unseen data: this is the subject of the next slide (validation set).

A validation set to detect overfitting

Validation set: a third subset of the data

Split the **training** set again, into a part used to fit and a part used to **validate**.



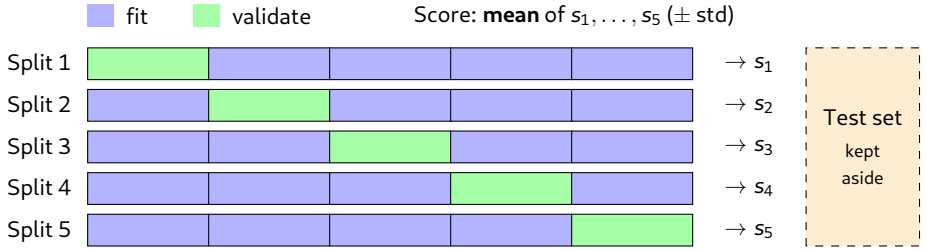


Why a third subset? Detecting overfitting requires measuring the error on data unseen during training (the orange curve of the previous slide). The test set cannot play this role: if you look at it to check for overfitting, and then change your model accordingly, you have indirectly tuned your model on it, and its score becomes optimistic! So the training set is split again: a part to fit the parameters, and a **validation set** to watch for overfitting (a validation error much higher than the training error is the warning sign). The test set is used once, at the very end (typical split: 60/20/20).

Cross-validation

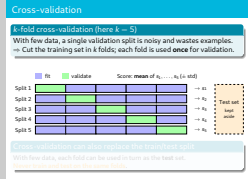
k-fold cross-validation (here $k = 5$)

With few data, a single validation split is noisy and wastes examples.
⇒ Cut the training set in k folds; each fold is used **once** for validation.



Cross-validation can also replace the train/test split

With few data, each fold can be used in turn as the **test** set.
Never train and test on the same folds.



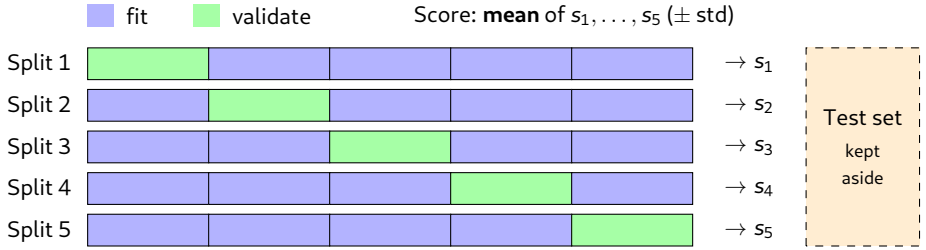
Cross-validation. With few data, the score on a single validation set depends a lot on which examples ended up in it, and these examples are never used to fit. In k-fold cross-validation, the model is trained k times, each time validated on a different fold: every example is used exactly once for validation. The mean of the k scores is more reliable, and their standard deviation says how much to trust it. The price: k trainings instead of one ($k = 5$ or 10 usually).
Cross-validation has two uses. Here, cross-validation replaces the validation set, to detect overfitting, and to choose hyperparameters. But it is also very often used to replace the train/test split, especially on small datasets: each fold is used in turn as the test set, and the mean test score is reported.
Careful: if you do cross-validation for both validation and testing, the folds must be different ("nested" cross-validation). Otherwise you are tuning on the test set, and the score is optimistic.

With imbalanced classes, use *stratified* folds (same class proportions in each fold). In `sklearn`: `cross_val_score`, and `GridSearchCV` to choose hyperparameters.

Cross-validation

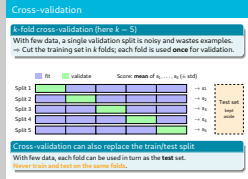
k-fold cross-validation (here $k = 5$)

With few data, a single validation split is noisy and wastes examples.
$\Rightarrow$ Cut the training set in k folds; each fold is used **once** for validation.



Cross-validation can also replace the train/test split

With few data, each fold can be used in turn as the **test** set.
Never train and test on the same folds.



Cross-validation. With few data, the score on a single validation set depends a lot on which examples ended up in it, and these examples are never used to fit. In k-fold cross-validation, the model is trained k times, each time validated on a different fold: every example is used exactly once for validation. The mean of the k scores is more reliable, and their standard deviation says how much to trust it. The price: k trainings instead of one ($k = 5$ or 10 usually).
Cross-validation has two uses. Here, cross-validation replaces the validation set, to detect overfitting, and to choose hyperparameters. But it is also very often used to replace the train/test split, especially on small datasets: each fold is used in turn as the test set, and the mean test score is reported.
Careful: if you do cross-validation for both validation and testing, the folds must be different ("nested" cross-validation). Otherwise you are tuning on the test set, and the score is optimistic.

With imbalanced classes, use *stratified* folds (same class proportions in each fold). In `sklearn`: `cross_val_score`, and `GridSearchCV` to choose hyperparameters.

High-dimensional data (1/2)

Curse of dimensionality

- Geometry is not intuitive in **high dimension**,
- Efficient methods in 2D are not necessarily still valid.

Random points in a cube: inside or outside the ball it contains

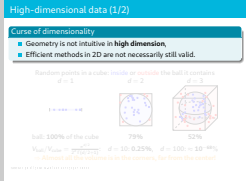
$d = 1$ $d = 2$ $d = 3$

ball: **100%** of the cube **79%** **52%**

$V_{\text{ball}}/V_{\text{cube}} = \frac{\pi^{d/2}}{2^d\,\Gamma(d/2+1)}$: $d = 10$: **0.25%**, $d = 100$: $\approx$ **10^{-68}%**

$\Rightarrow$ **Almost all the volume is in the corners, far from the center!**

see https://youtu.be/dZrGXYty3qc?t=533



The ball (a disc in 2D, a segment in 1D) touches the faces of the cube. Which fraction of the cube does it fill? 100% in 1D, 79% in 2D, 52% in 3D; the crosses, drawn uniformly in the cube, show the same trend (fewer and fewer blue ones). In 10D, the ball fills only 0.25% of the cube, and in 100D, 10^{-68}%. The volume concentrates in the 2^d corners, far from the center. Random points in high dimension are almost never near the center, and almost never near each other. The take-home message of this slide is that geometry is not intuitive in high dimension, and methods that work well in 2D may fail in 100D. Be careful the, because some algorithms lose their meaning (e.g., KNN: because all points become almost equally far, "nearest neighbours" lose their meaning).

You may use an analogy: finding lost keys on a 10 m line is relatively easy, in a 10 m wide field will be more complex, and in a 10 m high building hopeless: each dimension multiplies the space to explore, while the number of examples stays the same.

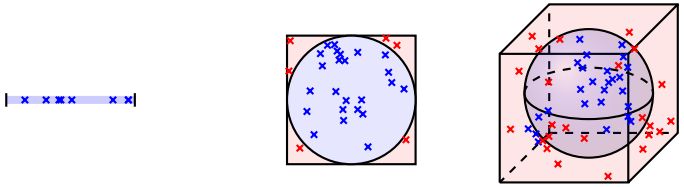
High-dimensional data (1/2)

Curse of dimensionality

- Geometry is not intuitive in **high dimension**,
- Efficient methods in 2D are not necessarily still valid.

Random points in a cube: inside or outside the ball it contains

$d = 1$ $d = 2$ $d = 3$

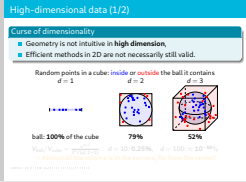


ball: **100%** of the cube **79%** **52%**

$V_{\text{ball}}/V_{\text{cube}} = \frac{\pi^{d/2}}{2^d\,\Gamma(d/2+1)}$: $d = 10$: **0.25%**, $d = 100$: $\approx$ **10^{-68}%**

$\Rightarrow$ **Almost all the volume is in the corners, far from the center!**

see https://youtu.be/dZrGXYty3qc?t=533



The ball (a disc in 2D, a segment in 1D) touches the faces of the cube. Which fraction of the cube does it fill? 100% in 1D, 79% in 2D, 52% in 3D; the crosses, drawn uniformly in the cube, show the same trend (fewer and fewer blue ones). In 10D, the ball fills only 0.25% of the cube, and in 100D, 10^{-68}%. The volume concentrates in the 2^d corners, far from the center. Random points in high dimension are almost never near the center, and almost never near each other. The take-home message of this slide is that geometry is not intuitive in high dimension, and methods that work well in 2D may fail in 100D. Be careful the, because some algorithms lose their meaning (e.g., KNN: because all points become almost equally far, "nearest neighbours" lose their meaning).

You may use an analogy: finding lost keys on a 10 m line is relatively easy, in a 10 m wide field will be more complex, and in a 10 m high building hopeless: each dimension multiplies the space to explore, while the number of examples stays the same.

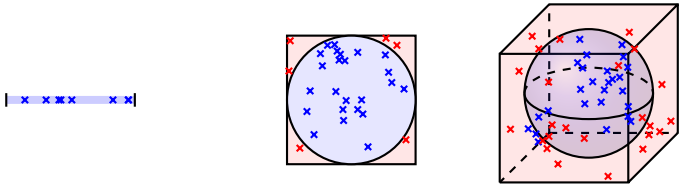
High-dimensional data (1/2)

Curse of dimensionality

- Geometry is not intuitive in **high dimension**,
- Efficient methods in 2D are not necessarily still valid.

Random points in a cube: inside or outside the ball it contains

$d = 1$ $d = 2$ $d = 3$

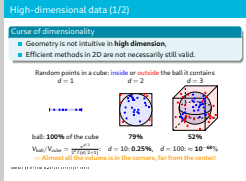


ball: **100%** of the cube **79%** **52%**

$V_{\text{ball}}/V_{\text{cube}} = \frac{\pi^{d/2}}{2^d \Gamma(d/2+1)}$: $d = 10$: **0.25%**, $d = 100$: $\approx$ **10^{-68}%**

$\Rightarrow$ **Almost all the volume is in the corners, far from the center!**

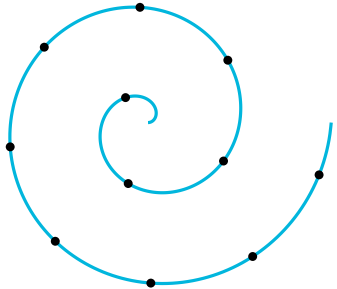
see https://youtu.be/dZrGXYty3qc?t=533



The ball (a disc in 2D, a segment in 1D) touches the faces of the cube. Which fraction of the cube does it fill? 100% in 1D, 79% in 2D, 52% in 3D; the crosses, drawn uniformly in the cube, show the same trend (fewer and fewer blue ones). In 10D, the ball fills only 0.25% of the cube, and in 100D, 10^{-68}%. The volume concentrates in the 2^d corners, far from the center. Random points in high dimension are almost never near the center, and almost never near each other. The take-home message of this slide is that geometry is not intuitive in high dimension, and methods that work well in 2D may fail in 100D. Be careful the, because some algorithms lose their meaning (e.g., KNN: because all points become almost equally far, "nearest neighbours" lose their meaning).

You may use an analogy: finding lost keys on a 10 m line is relatively easy, in a 10 m wide field will be more complex, and in a 10 m high building hopeless: each dimension multiplies the space to explore, while the number of examples stays the same.

High-dimensional data (2/2): data lives on manifolds



original space ($\mathbb{R}^d$)

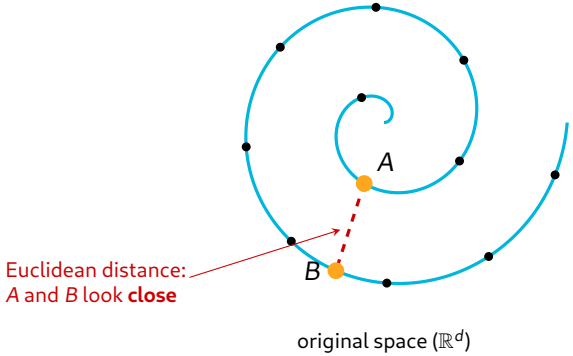
High-dimensional data (2/2): data lives on manifolds



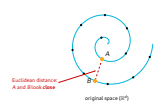
We further present the "swiss roll" example, that introduces pretty easily a useful idea: data may be concentrated near a lower dimensional structure, and the natural space of data may not always be suited to represent it! Here, the data is in $\mathbb{R}^2$, but it is concentrated on a 1D spiral.
The consequence is that distances in the original space are not meaningful: two points on neighbouring layers of the spiral are close in the Euclidean sense, but far along the spiral. A method relying on the naive distance (KNN typically) would consider them similar, and be wrong.

Finding this structure (the "manifold") is a powerful way to solve the curse of dimensionality, and is the main motivation behind feature descriptions and embeddings

High-dimensional data (2/2): data lives on manifolds



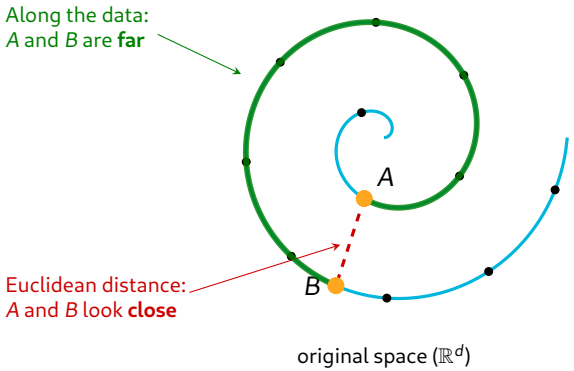
High-dimensional data (2/2): data lives on manifolds

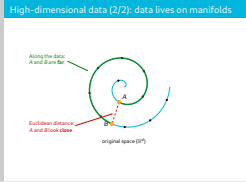


We further present the "swiss roll" example, that introduces pretty easily a useful idea: data may be concentrated near a lower dimensional structure, and the natural space of data may not always be suited to represent it! Here, the data is in $\mathbb{R}^2$, but it is concentrated on a 1D spiral.
The consequence is that distances in the original space are not meaningful: two points on neighbouring layers of the spiral are close in the Euclidean sense, but far along the spiral. A method relying on the naive distance (KNN typically) would consider them similar, and be wrong.

Finding this structure (the "manifold") is a powerful way to solve the curse of dimensionality, and is the main motivation behind feature descriptions and embeddings

High-dimensional data (2/2): data lives on manifolds

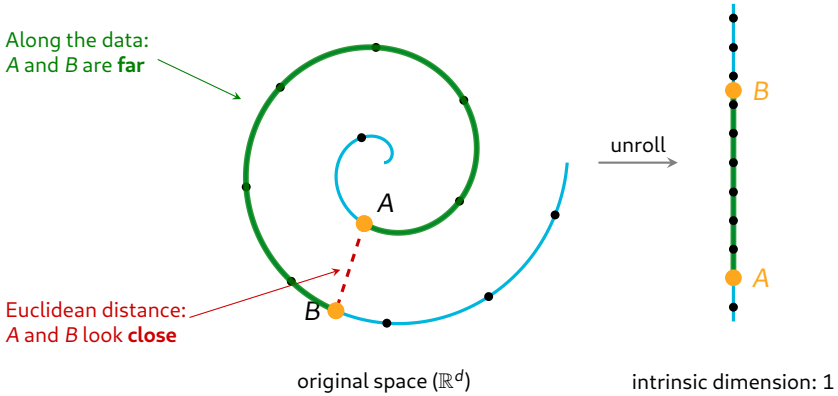




We further present the "swiss roll" example, that introduces pretty easily a useful idea: data may be concentrated near a lower dimensional structure, and the natural space of data may not always be suited to represent it! Here, the data is in $\mathbb{R}^2$, but it is concentrated on a 1D spiral.
The consequence is that distances in the original space are not meaningful: two points on neighbouring layers of the spiral are close in the Euclidean sense, but far along the spiral. A method relying on the naive distance (KNN typically) would consider them similar, and be wrong.

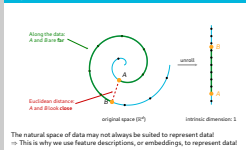
Finding this structure (the "manifold") is a powerful way to solve the curse of dimensionality, and is the main motivation behind feature descriptions and embeddings

High-dimensional data (2/2): data lives on manifolds



The natural space of data may not always be suited to represent data!
⇒ This is why we use feature descriptions, or embeddings, to represent data!

High-dimensional data (2/2): data lives on manifolds



We further present the "swiss roll" example, that introduces pretty easily a useful idea: data may be concentrated near a lower dimensional structure, and the natural space of data may not always be suited to represent it! Here, the data is in $\mathbb{R}^2$, but it is concentrated on a 1D spiral.
The consequence is that distances in the original space are not meaningful: two points on neighbouring layers of the spiral are close in the Euclidean sense, but far along the spiral. A method relying on the naive distance (KNN typically) would consider them similar, and be wrong.

Finding this structure (the "manifold") is a powerful way to solve the curse of dimensionality, and is the main motivation behind feature descriptions and embeddings

Time and material resources

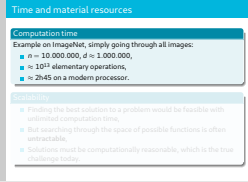
Computation time

Example on ImageNet, simply going through all images:

- $n = 10.000.000$, $d \approx 1.000.000$,
- $\approx 10^{13}$ elementary operations,
- $\approx$ 2h45 on a modern processor.

Scalability

- Finding the best solution to a problem would be feasible with unlimited computation time,
- But searching through the space of possible functions is often **untractable**,
- Solutions must be computationally reasonable, which is the true challenge today.



The cost of the data. For ImageNet: about 10^7 images of roughly 10^6 values, so 10^{13} operations just to *read* every value once (around three hours, for one single pass doing nothing!). Training needs many passes, each far more than one operation per value. Dataset size is a hard practical constraint, which is why your project dataset should stay in the 1 000–10 000 range.

The cost of the search. Learning selects a function among all those the model can represent, and that set grows extremely fast with the number of parameters: exhaustive search is impossible. So we need an heuristic: an automatic procedure able to find a "good enough" solution (such as gradient descent, which finds a local optimum with no guarantee of being global).

Hence the message: a method is judged on the quality it reaches **within an acceptable computational budget**. Reflex for the lab: try your training on a small subsample before launching it on everything.

Time and material resources

Computation time

Example on ImageNet, simply going through all images:

- $n = 10.000.000$, $d \approx 1.000.000$,
- $\approx 10^{13}$ elementary operations,
- $\approx$ 2h45 on a modern processor.

Scalability

- Finding the best solution to a problem would be feasible with unlimited computation time,
- But searching through the space of possible functions is often **untractable**,
- Solutions must be computationally reasonable, which is the true challenge today.

Time and material resources
Computation time
Example on ImageNet, simply going through all images:
- $n = 10.000.000$, $d \approx 1.000.000$,
- $\approx 10^{13}$ elementary operations,
- $\approx$ 2h45 on a modern processor.
Scalability
- Finding the best solution to a problem would be feasible with unlimited computation time,
- But searching through the space of possible functions is often **untractable**,
- Solutions must be computationally reasonable, which is the true challenge today.

The cost of the data. For ImageNet: about 10^7 images of roughly 10^6 values, so 10^{13} operations just to *read* every value once (around three hours, for one single pass doing nothing!). Training needs many passes, each far more than one operation per value. Dataset size is a hard practical constraint, which is why your project dataset should stay in the 1 000–10 000 range.

The cost of the search. Learning selects a function among all those the model can represent, and that set grows extremely fast with the number of parameters: exhaustive search is impossible. So we need an heuristic: an automatic procedure able to find a "good enough" solution (such as gradient descent, which finds a local optimum with no guarantee of being global).

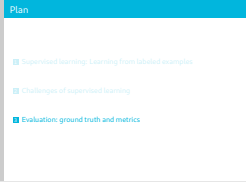
Hence the message: a method is judged on the quality it reaches **within an acceptable computational budget**. Reflex for the lab: try your training on a small subsample before launching it on everything.

Plan

1 Supervised learning: Learning from labeled examples

2 Challenges of supervised learning

3 Evaluation: ground truth and metrics



Evaluation: ground truth vs. estimation

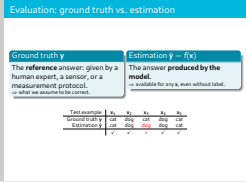
Ground truth $\mathbf{y}$

The **reference** answer: given by a human expert, a sensor, or a measurement protocol.
⇒ what we *assume* to be correct.

Estimation $\hat{\mathbf{y}} = f(\mathbf{x})$

The answer **produced by the model**.
⇒ available for any $\mathbf{x}$, even without label.

Test example	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$
Ground truth $\mathbf{y}$	cat	dog	cat	dog	cat
Estimation $\hat{\mathbf{y}}$	cat	dog	dog	dog	cat
	✓	✓	×	✓	✓



Two objects never to be confused.
Ground truth $\mathbf{y}$ is the label attached to an example: an annotator wrote "cat" under this image, a doctor wrote "benign" in this report. It exists only for the examples we have annotated, and producing it is usually the most expensive part of a project.
Estimation $\hat{\mathbf{y}} = f(\mathbf{x})$ is what the model outputs. It exists for *any* input, including inputs nobody ever labeled (which is precisely what makes the model useful).
Evaluation puts the two side by side and counts. The important score is the evaluation performed on the **test set**, because the model has already seen the training set, and thus the answers.

Finally, the ground truth is **not** the truth: annotators disagree, labels contain errors, many tasks are ambiguous. This caps the performance any model can reach, and a model at 100% is more often the sign of a bug (typically a test set leak) than of a perfect model.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.
$\Rightarrow$ we need to **summarise** the comparison into a few numbers.

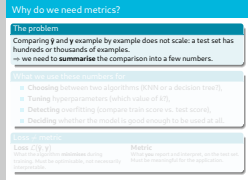
What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric
What **you** report and interpret, on the test set. Must be meaningful for the application.



A metric takes the whole list of pairs $(\hat{\mathbf{y}}_i, \mathbf{y}_i)$ on the test set and returns a few scalars. Without it you would inspect every prediction by hand, impossible beyond a few dozen examples, with no way to say that model A beats model B. Note that a single number is generally meaningless in isolation (How good is *85%?*). What informs is a **comparison**: train vs. test (overfitting), model A vs. model B, or model A vs. a trivial baseline such as "always predict the majority class". Always compute that baseline first: if your classifier does not beat it, it has learned nothing.

Confusing **Loss** and **metric** is very common. The loss is an *optimisation* tool: cross-entropy is used because it is smooth and differentiable, but "a cross-entropy of 0.31" means nothing on its own. The metric is a *communication* tool: "the model detects 92% of the faulty parts" is actionable in real life. They are computed on different data as well. Consequence: minimising the loss does not automatically maximise the metric you care about.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.
$\Rightarrow$ we need to **summarise** the comparison into a few numbers.

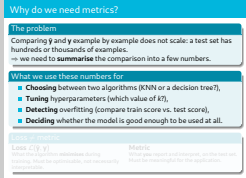
What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric
What **you** report and interpret, on the test set. Must be meaningful for the application.



A metric takes the whole list of pairs $(\hat{\mathbf{y}}_i, \mathbf{y}_i)$ on the test set and returns a few scalars. Without it you would inspect every prediction by hand, impossible beyond a few dozen examples, with no way to say that model A beats model B. Note that a single number is generally meaningless in isolation (How good is *85%?*). What informs is a **comparison**: train vs. test (overfitting), model A vs. model B, or model A vs. a trivial baseline such as "always predict the majority class". Always compute that baseline first: if your classifier does not beat it, it has learned nothing.

Confusing **Loss** and **metric** is very common. The loss is an *optimisation* tool: cross-entropy is used because it is smooth and differentiable, but "a cross-entropy of 0.31" means nothing on its own. The metric is a *communication* tool: "the model detects 92% of the faulty parts" is actionable in real life. They are computed on different data as well. Consequence: minimising the loss does not automatically maximise the metric you care about.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.
$\Rightarrow$ we need to **summarise** the comparison into a few numbers.

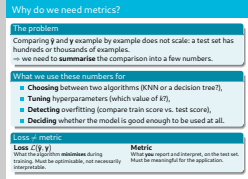
What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric
What **you** report and interpret, on the test set. Must be meaningful for the application.



A metric takes the whole list of pairs $(\hat{\mathbf{y}}_i, \mathbf{y}_i)$ on the test set and returns a few scalars. Without it you would inspect every prediction by hand, impossible beyond a few dozen examples, with no way to say that model A beats model B. Note that a single number is generally meaningless in isolation (How good is *85%?*). What informs is a **comparison**: train vs. test (overfitting), model A vs. model B, or model A vs. a trivial baseline such as "always predict the majority class". Always compute that baseline first: if your classifier does not beat it, it has learned nothing.

Confusing **Loss** and **metric** is very common. The loss is an *optimisation* tool: cross-entropy is used because it is smooth and differentiable, but "a cross-entropy of 0.31" means nothing on its own. The metric is a *communication* tool: "the model detects 92% of the faulty parts" is actionable in real life. They are computed on different data as well. Consequence: minimising the loss does not automatically maximise the metric you care about.

A first metric: accuracy

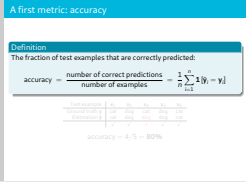
Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\left[\hat{\mathbf{y}}_i = \mathbf{y}_i\right]$$

Test example	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$
Ground truth $\mathbf{y}$	cat	dog	cat	dog	cat
Estimation $\hat{\mathbf{y}}$	cat	dog	dog	dog	cat
	✓	✓	×	✓	✓

$$\text{accuracy} = 4/5 = \mathbf{80\%}$$



The simplest and most widely used metric: count the correct predictions and divide by the number of examples. The notation $\mathbf{1}[\cdot]$ is the indicator function, worth 1 when the condition inside is true and 0 otherwise, so the sum is just the number of correct answers.

It ranges from 0 to 1 (often given in %), and it is the quantity returned by `score` on any `sklearn` classifier, so you will meet it constantly in the lab.

Two reflexes before reading any accuracy. First, **on which set** was it computed? A training accuracy and a test accuracy tell completely different stories, and the gap between them is your overfitting indicator. Second, **what would a trivial model score?** With two balanced classes, random guessing already gives 50%, so 55% is nearly worthless; a relevant baseline is "always predicting the majority class".

Accuracy also assumes that all mistakes are equally serious and that all classes matter equally. This is actually frequently false, and other metrics are generally needed to get a complete picture of the model's performance.

A first metric: accuracy

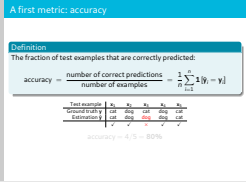
Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\left[\hat{\mathbf{y}}_i = \mathbf{y}_i\right]$$

Test example	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$
Ground truth $\mathbf{y}$	cat	dog	cat	dog	cat
Estimation $\hat{\mathbf{y}}$	cat	dog	dog	dog	cat
	✓	✓	×	✓	✓

$$\text{accuracy} = 4/5 = \mathbf{80\%}$$



The simplest and most widely used metric: count the correct predictions and divide by the number of examples. The notation $\mathbf{1}[\cdot]$ is the indicator function, worth 1 when the condition inside is true and 0 otherwise, so the sum is just the number of correct answers.

It ranges from 0 to 1 (often given in %), and it is the quantity returned by `score` on any `sklearn` classifier, so you will meet it constantly in the lab.

Two reflexes before reading any accuracy. First, **on which set** was it computed? A training accuracy and a test accuracy tell completely different stories, and the gap between them is your overfitting indicator. Second, **what would a trivial model score?** With two balanced classes, random guessing already gives 50%, so 55% is nearly worthless; a relevant baseline is "always predicting the majority class".

Accuracy also assumes that all mistakes are equally serious and that all classes matter equally. This is actually frequently false, and other metrics are generally needed to get a complete picture of the model's performance.

A first metric: accuracy

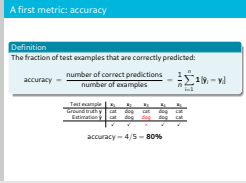
Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\left[\hat{\mathbf{y}}_i = \mathbf{y}_i\right]$$

Test example	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$
Ground truth $\mathbf{y}$	cat	dog	cat	dog	cat
Estimation $\hat{\mathbf{y}}$	cat	dog	dog	dog	cat
	✓	✓	×	✓	✓

$$\text{accuracy} = 4/5 = \mathbf{80\%}$$



The simplest and most widely used metric: count the correct predictions and divide by the number of examples. The notation $\mathbf{1}[\cdot]$ is the indicator function, worth 1 when the condition inside is true and 0 otherwise, so the sum is just the number of correct answers.

It ranges from 0 to 1 (often given in %), and it is the quantity returned by `score` on any `sklearn` classifier, so you will meet it constantly in the lab.

Two reflexes before reading any accuracy. First, **on which set** was it computed? A training accuracy and a test accuracy tell completely different stories, and the gap between them is your overfitting indicator. Second, **what would a trivial model score?** With two balanced classes, random guessing already gives 50%, so 55% is nearly worthless; a relevant baseline is "always predicting the majority class".

Accuracy also assumes that all mistakes are equally serious and that all classes matter equally. This is actually frequently false, and other metrics are generally needed to get a complete picture of the model's performance.

Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.

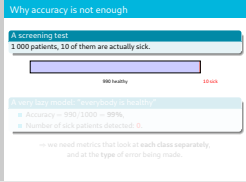


990 healthy 10 sick

A very lazy model: "everybody is healthy"

- Accuracy = 990/1000 = **99%**,
- Number of sick patients detected: **0**.

⇒ we need metrics that look at **each class separately**, and at the **type** of error being made.



Note the message here: **accuracy may not be enough**
When one class dominates ("class imbalance"), accuracy is dominated by it. The "everybody is healthy" model is useless, yet it scores 99%. Realistic tasks are very often imbalanced: fraud, rare disease screening, defect detection, spam. Your own project data probably is too; check the class counts before interpreting any score.
In addition, **errors are not symmetric!** Accuracy weights all mistakes equally, but these two have nothing in common. Telling a healthy patient he may be sick (*false positive*) costs an extra examination; telling a sick patient he is healthy (*false negative*) may cost his life. A good evaluation shows them separately.

Both observations lead to the next slides: precision and recall each look at one type of error, and the confusion matrix shows them all. In the lab you get both with `classification_report` and `confusion_matrix`.

Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.

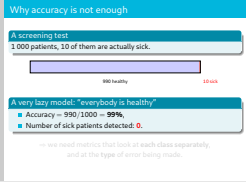


990 healthy 10 sick

A very lazy model: “everybody is healthy”

- Accuracy $= 990/1000 =$ **99%**,
- Number of sick patients detected: **0**.

$\Rightarrow$ we need metrics that look at **each class separately**, and at the **type** of error being made.



Note the message here: **accuracy may not be enough**
When one class dominates (“class imbalance”), accuracy is dominated by it. The “everybody is healthy” model is useless, yet it scores 99%. Realistic tasks are very often imbalanced: fraud, rare disease screening, defect detection, spam. Your own project data probably is too; check the class counts before interpreting any score.
In addition, **errors are not symmetric!** Accuracy weights all mistakes equally, but these two have nothing in common. Telling a healthy patient he may be sick (*false positive*) costs an extra examination; telling a sick patient he is healthy (*false negative*) may cost his life. A good evaluation shows them separately.

Both observations lead to the next slides: precision and recall each look at one type of error, and the confusion matrix shows them all. In the lab you get both with `classification_report` and `confusion_matrix`.

Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.

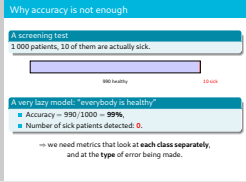


990 healthy 10 sick

A very lazy model: "everybody is healthy"

- Accuracy $= 990/1000 =$ **99%**,
- Number of sick patients detected: **0**.

$\Rightarrow$ we need metrics that look at **each class separately**, and at the **type** of error being made.



Note the message here: **accuracy may not be enough**
When one class dominates ("class imbalance"), accuracy is dominated by it. The "everybody is healthy" model is useless, yet it scores 99%. Realistic tasks are very often imbalanced: fraud, rare disease screening, defect detection, spam. Your own project data probably is too; check the class counts before interpreting any score.
In addition, **errors are not symmetric!** Accuracy weights all mistakes equally, but these two have nothing in common. Telling a healthy patient he may be sick (*false positive*) costs an extra examination; telling a sick patient he is healthy (*false negative*) may cost his life. A good evaluation shows them separately.
Both observations lead to the next slides: precision and recall each look at one type of error, and the confusion matrix shows them all. In the lab you get both with `classification_report` and `confusion_matrix`.

Precision and recall

Definition (for **one** class, chosen as "positive")

$$\text{precision} = \frac{TP}{TP + FP} \qquad \text{recall} = \frac{TP}{TP + FN}$$

Among the examples **predicted** positive, how many really are?

Among the **truly** positive examples, how many are found?

TP: true positives, *FP*: false positives (*false alarms*), *FN*: false negatives (*missed*).

Test example	x_1	x_2	x_3	x_4	x_5
Ground truth y	cat	dog	cat	dog	cat
Estimation $\hat{y}$	cat	dog	dog	dog	cat

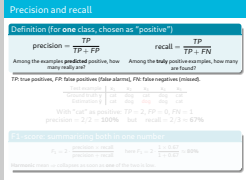
With "cat" as positive: $TP = 2$, $FP = 0$, $FN = 1$

precision $= 2/2 =$ **100%** but recall $= 2/3 \approx$ **67%**

F1-score: summarising both in one number

$$F_1 = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \qquad \text{here } F_1 = 2 \cdot \frac{1 \times 0.67}{1 + 0.67} \approx \textbf{80\%}$$

Harmonic mean $\Rightarrow$ collapses as soon as **one** of the two is low.



Each of the two isolates one type of error, which is exactly what accuracy failed to do.

Precision is penalised by **false alarms** (*FP*): the model shouted "cat" when there was none. Recall is penalised by **misses** (*FN*): a cat went unnoticed. Here the model never claims a cat wrongly (precision 100%) but lets one cat slip through (recall 67%).

Both are defined **relative to a chosen positive class**. Swap the roles and the numbers change: taking "dog" as positive gives a precision of $2/3$ and a recall of $2/2$ on the very same predictions.

The two are in tension: declaring everybody positive gives a recall of 100% and a terrible precision. Hence they are reported **together**, often summarised by the **F1-score**, their harmonic mean $2PR/(P+R)$, low as soon as *one* of the two is low. Which one matters most is an application decision: a medical application favours recall, a spam filter favours precision.

Precision and recall

Definition (for **one** class, chosen as "positive")

$$\text{precision} = \frac{TP}{TP + FP} \qquad \text{recall} = \frac{TP}{TP + FN}$$

Among the examples **predicted** positive, how many really are?

Among the **truly** positive examples, how many are found?

TP: true positives, *FP*: false positives (*false alarms*), *FN*: false negatives (*missed*).

Test example	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$
Ground truth $\mathbf{y}$	cat	dog	cat	dog	cat
Estimation $\hat{\mathbf{y}}$	cat	dog	dog	dog	cat

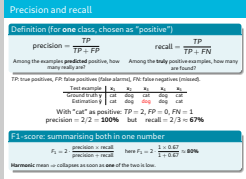
With "cat" as positive: $TP = 2$, $FP = 0$, $FN = 1$

precision $= 2/2 =$ **100%** but recall $= 2/3 \approx$ **67%**

F1-score: summarising both in one number

$$F_1 = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \qquad \text{here } F_1 = 2 \cdot \frac{1 \times 0.67}{1 + 0.67} \approx \mathbf{80\%}$$

Harmonic mean $\Rightarrow$ collapses as soon as **one** of the two is low.



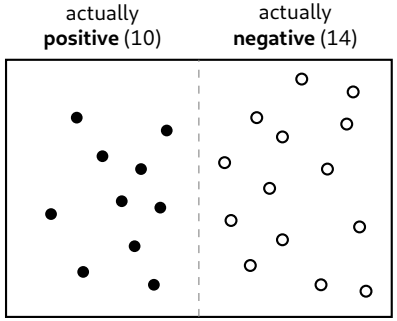
Each of the two isolates one type of error, which is exactly what accuracy failed to do.

Precision is penalised by **false alarms** (*FP*): the model shouted "cat" when there was none. Recall is penalised by **misses** (*FN*): a cat went unnoticed. Here the model never claims a cat wrongly (precision 100%) but lets one cat slip through (recall 67%).

Both are defined **relative to a chosen positive class**. Swap the roles and the numbers change: taking "dog" as positive gives a precision of $2/3$ and a recall of $2/2$ on the very same predictions.

The two are in tension: declaring everybody positive gives a recall of 100% and a terrible precision. Hence they are reported **together**, often summarised by the **F1-score**, their harmonic mean $2PR/(P+R)$, low as soon as *one* of the two is low. Which one matters most is an application decision: a medical application favours recall, a spam filter favours precision.

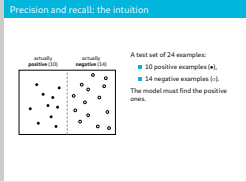
Precision and recall: the intuition



A test set of 24 examples:

- 10 positive examples (●),
- 14 negative examples (○).

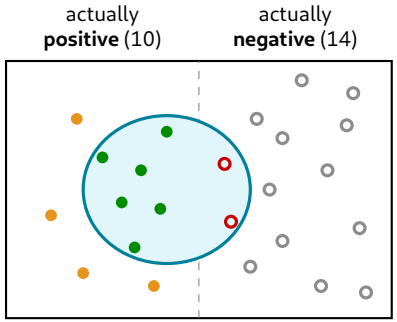
The model must find the positive ones.



The left half contains the examples that *are* positive (filled dots), the right half the negatives (hollow dots). The **ellipse** contains the examples the model *declared* positive. Each example falls into one of four cases: *TP* (green), *FN* (orange), *FP* (red), *TN* (grey).
Precision looks *inside the ellipse*: which fraction of what the model declared positive really is? **Recall** looks at the *left half*: which fraction of the positives was caught?
The last two overlays show two other models on the same data. A cautious model (small ellipse, only the sure cases) makes no false alarm, precision 100%, but misses 7 positives out of 10, recall 30%. A permissive model (large ellipse) catches all positives, recall 100%, but raises 7 false alarms, precision 59%. Hence the two metrics pull in opposite directions and are always reported together.

Accuracy also counts the true negatives outside the ellipse. When negatives vastly outnumber positives, they dominate the count, e.g. in the previous example with the 99% screening model seen before.

Precision and recall: the intuition



A first model

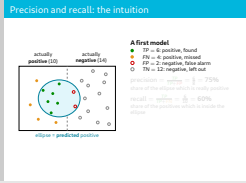
- $TP = 6$: positive, found
- $FN = 4$: positive, missed
- $FP = 2$: negative, false alarm
- $TN = 12$: negative, left out

$$\text{precision} = \frac{TP}{TP+FP} = \frac{6}{8} = \mathbf{75\%}$$

share of the ellipse which is really positive

$$\text{recall} = \frac{TP}{TP+FN} = \frac{6}{10} = \mathbf{60\%}$$

share of the positives which is inside the ellipse



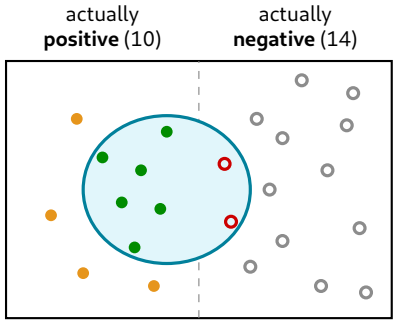
The left half contains the examples that *are* positive (filled dots), the right half the negatives (hollow dots). The **ellipse** contains the examples the model *declared* positive. Each example falls into one of four cases: *TP* (green), *FN* (orange), *FP* (red), *TN* (grey).

Precision looks *inside the ellipse*: which fraction of what the model declared positive really is? **Recall** looks at the *left half*: which fraction of the positives was caught?

The last two overlays show two other models on the same data. A cautious model (small ellipse, only the sure cases) makes no false alarm, precision 100%, but misses 7 positives out of 10, recall 30%. A permissive model (large ellipse) catches all positives, recall 100%, but raises 7 false alarms, precision 59%. Hence the two metrics pull in opposite directions and are always reported together.

Accuracy also counts the true negatives outside the ellipse. When negatives vastly outnumber positives, they dominate the count, e.g. in the previous example with the 99% screening model seen before.

Precision and recall: the intuition

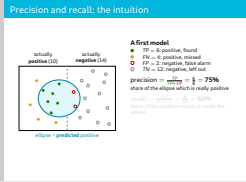


A first model

- $TP = 6$: positive, found
- $FN = 4$: positive, missed
- $FP = 2$: negative, false alarm
- $TN = 12$: negative, left out

precision $= \frac{TP}{TP+FP} = \frac{6}{8} =$ **75%**
share of the ellipse which is really positive

recall $= \frac{TP}{TP+FN} = \frac{6}{10} =$ **60%**
share of the positives which is inside the ellipse



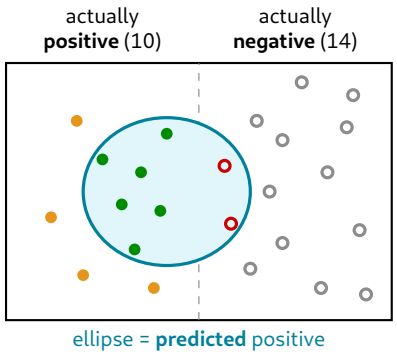
The left half contains the examples that *are* positive (filled dots), the right half the negatives (hollow dots). The **ellipse** contains the examples the model *declared* positive. Each example falls into one of four cases: *TP* (green), *FN* (orange), *FP* (red), *TN* (grey).

Precision looks *inside the ellipse*: which fraction of what the model declared positive really is? **Recall** looks at the *left half*: which fraction of the positives was caught?

The last two overlays show two other models on the same data. A cautious model (small ellipse, only the sure cases) makes no false alarm, precision 100%, but misses 7 positives out of 10, recall 30%. A permissive model (large ellipse) catches all positives, recall 100%, but raises 7 false alarms, precision 59%. Hence the two metrics pull in opposite directions and are always reported together.

Accuracy also counts the true negatives outside the ellipse. When negatives vastly outnumber positives, they dominate the count, e.g. in the previous example with the 99% screening model seen before.

Precision and recall: the intuition

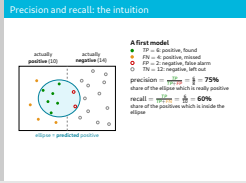


A first model

- $TP = 6$: positive, found
- $FN = 4$: positive, missed
- $FP = 2$: negative, false alarm
- $TN = 12$: negative, left out

precision $= \frac{TP}{TP+FP} = \frac{6}{8} =$ **75%**
share of the ellipse which is really positive

recall $= \frac{TP}{TP+FN} = \frac{6}{10} =$ **60%**
share of the positives which is inside the ellipse



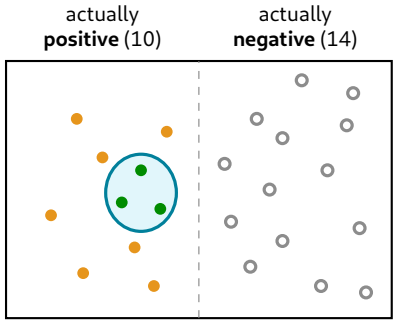
The left half contains the examples that *are* positive (filled dots), the right half the negatives (hollow dots). The **ellipse** contains the examples the model *declared* positive. Each example falls into one of four cases: *TP* (green), *FN* (orange), *FP* (red), *TN* (grey).

Precision looks *inside the ellipse*: which fraction of what the model declared positive really is? **Recall** looks at the *left half*: which fraction of the positives was caught?

The last two overlays show two other models on the same data. A cautious model (small ellipse, only the sure cases) makes no false alarm, precision 100%, but misses 7 positives out of 10, recall 30%. A permissive model (large ellipse) catches all positives, recall 100%, but raises 7 false alarms, precision 59%. Hence the two metrics pull in opposite directions and are always reported together.

Accuracy also counts the true negatives outside the ellipse. When negatives vastly outnumber positives, they dominate the count, e.g. in the previous example with the 99% screening model seen before.

Precision and recall: the intuition



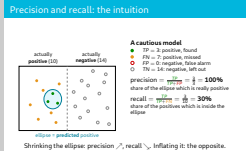
A cautious model

- $TP = 3$: positive, found
- $FN = 7$: positive, missed
- $FP = 0$: negative, false alarm
- $TN = 14$: negative, left out

$\text{precision} = \frac{TP}{TP+FP} = \frac{3}{3} = \mathbf{100\%}$
share of the ellipse which is really positive

$\text{recall} = \frac{TP}{TP+FN} = \frac{3}{10} = \mathbf{30\%}$
share of the positives which is inside the ellipse

Shrinking the ellipse: precision ↗, recall ↘. Inflating it: the opposite.



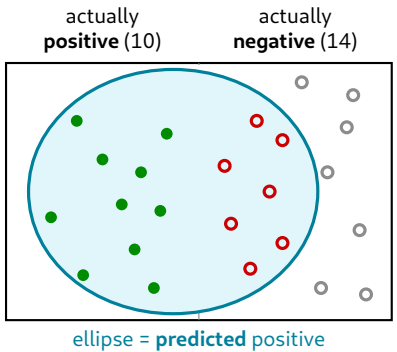
The left half contains the examples that *are* positive (filled dots), the right half the negatives (hollow dots). The **ellipse** contains the examples the model *declared* positive. Each example falls into one of four cases: *TP* (green), *FN* (orange), *FP* (red), *TN* (grey).

Precision looks *inside the ellipse*: which fraction of what the model declared positive really is? **Recall** looks at the *left half*: which fraction of the positives was caught?

The last two overlays show two other models on the same data. A cautious model (small ellipse, only the sure cases) makes no false alarm, precision 100%, but misses 7 positives out of 10, recall 30%. A permissive model (large ellipse) catches all positives, recall 100%, but raises 7 false alarms, precision 59%. Hence the two metrics pull in opposite directions and are always reported together.

Accuracy also counts the true negatives outside the ellipse. When negatives vastly outnumber positives, they dominate the count, e.g. in the previous example with the 99% screening model seen before.

Precision and recall: the intuition



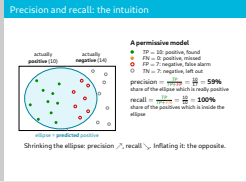
A permissive model

- $TP = 10$: positive, found
- $FN = 0$: positive, missed
- $FP = 7$: negative, false alarm
- $TN = 7$: negative, left out

precision $= \frac{TP}{TP+FP} = \frac{10}{17} =$ **59%**
share of the ellipse which is really positive

recall $= \frac{TP}{TP+FN} = \frac{10}{10} =$ **100%**
share of the positives which is inside the ellipse

Shrinking the ellipse: precision ↗, recall ↘. Inflating it: the opposite.



The left half contains the examples that *are* positive (filled dots), the right half the negatives (hollow dots). The **ellipse** contains the examples the model *declared* positive. Each example falls into one of four cases: *TP* (green), *FN* (orange), *FP* (red), *TN* (grey).

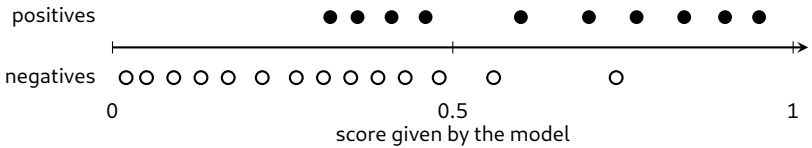
Precision looks *inside the ellipse*: which fraction of what the model declared positive really is? **Recall** looks at the *left half*: which fraction of the positives was caught?

The last two overlays show two other models on the same data. A cautious model (small ellipse, only the sure cases) makes no false alarm, precision 100%, but misses 7 positives out of 10, recall 30%. A permissive model (large ellipse) catches all positives, recall 100%, but raises 7 false alarms, precision 59%. Hence the two metrics pull in opposite directions and are always reported together.

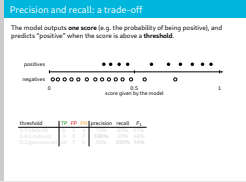
Accuracy also counts the true negatives outside the ellipse. When negatives vastly outnumber positives, they dominate the count, e.g. in the previous example with the 99% screening model seen before.

Precision and recall: a trade-off

The model outputs **one score** (e.g. the probability of being positive), and predicts "positive" when the score is above a **threshold**.



threshold	*TP*	*FP*	*FN*	precision	recall	F_1
0.5 (default)	6	2	4	75%	60%	67%
0.8 (cautious)	3	0	7	**100%**	30%	46%
0.3 (permissive)	10	7	0	59%	**100%**	**74%**



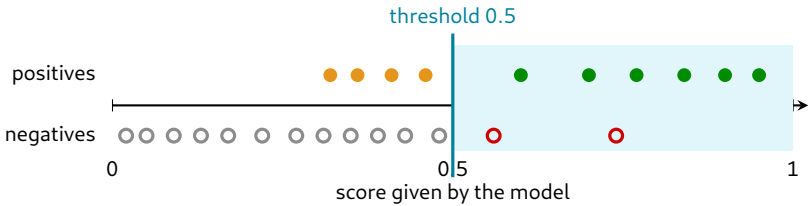
Most classifiers output **one score** (for instance a probability), turned into a class with a threshold (0.5 for instance). Each dot is a test example placed according to its score, positives on top, negatives below; everything right of the threshold is predicted positive. Raising the threshold to 0.8 removes all false alarms (precision 100%) but misses 7 positives (recall 30%). Lowering it to 0.3 catches every positive (recall 100%) with 7 false alarms (precision 59%). The threshold trades precision for recall *without retraining*; all possible thresholds draw the **precision/recall curve**.

The F_1 score summarises each line in one number: here it prefers the permissive threshold (74%), but it weights precision and recall equally, which is not always what the application needs.

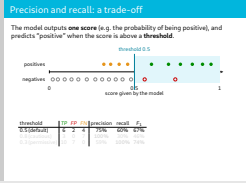
Where to stand is an application decision: a medical screening favours recall (a missed patient is worse than an extra examination), a spam filter favours precision (a lost email is worse than a spam). The threshold is a hyperparameter: choose it on validation data, **never on the test set**. In `sklearn`: `predict_proba` and `precision_recall_curve`.

Precision and recall: a trade-off

The model outputs **one score** (e.g. the probability of being positive), and predicts "positive" when the score is above a **threshold**.



threshold	*TP*	*FP*	*FN*	precision	recall	F_1
0.5 (default)	6	2	4	75%	60%	67%
0.8 (cautious)	3	0	7	**100%**	30%	46%
0.3 (permissive)	10	7	0	59%	**100%**	**74%**



Most classifiers output **one score** (for instance a probability), turned into a class with a threshold (0.5 for instance). Each dot is a test example placed according to its score, positives on top, negatives below; everything right of the threshold is predicted positive. Raising the threshold to 0.8 removes all false alarms (precision 100%) but misses 7 positives (recall 30%). Lowering it to 0.3 catches every positive (recall 100%) with 7 false alarms (precision 59%). The threshold trades precision for recall *without retraining*; all possible thresholds draw the **precision/recall curve**.

The F_1 score summarises each line in one number: here it prefers the permissive threshold (74%), but it weights precision and recall equally, which is not always what the application needs.

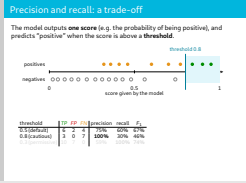
Where to stand is an application decision: a medical screening favours recall (a missed patient is worse than an extra examination), a spam filter favours precision (a lost email is worse than a spam). The threshold is a hyperparameter: choose it on validation data, **never on the test set**. In `sklearn`: `predict_proba` and `precision_recall_curve`.

Precision and recall: a trade-off

The model outputs **one score** (e.g. the probability of being positive), and predicts "positive" when the score is above a **threshold**.



threshold	*TP*	*FP*	*FN*	precision	recall	F_1
0.5 (default)	6	2	4	75%	60%	67%
0.8 (cautious)	3	0	7	**100%**	30%	46%
0.3 (permissive)	10	7	0	59%	**100%**	**74%**



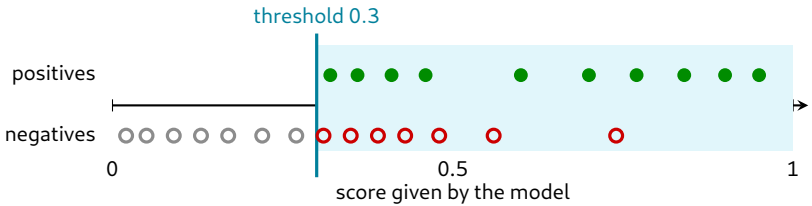
Most classifiers output **one score** (for instance a probability), turned into a class with a threshold (0.5 for instance). Each dot is a test example placed according to its score, positives on top, negatives below; everything right of the threshold is predicted positive. Raising the threshold to 0.8 removes all false alarms (precision 100%) but misses 7 positives (recall 30%). Lowering it to 0.3 catches every positive (recall 100%) with 7 false alarms (precision 59%). The threshold trades precision for recall *without retraining*; all possible thresholds draw the **precision/recall curve**.

The F_1 score summarises each line in one number: here it prefers the permissive threshold (74%), but it weights precision and recall equally, which is not always what the application needs.

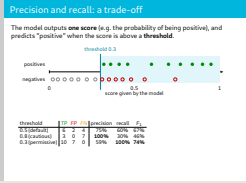
Where to stand is an application decision: a medical screening favours recall (a missed patient is worse than an extra examination), a spam filter favours precision (a lost email is worse than a spam). The threshold is a hyperparameter: choose it on validation data, **never on the test set**. In `sklearn`: `predict_proba` and `precision_recall_curve`.

Precision and recall: a trade-off

The model outputs **one score** (e.g. the probability of being positive), and predicts "positive" when the score is above a **threshold**.



threshold	*TP*	*FP*	*FN*	precision	recall	F_1
0.5 (default)	6	2	4	75%	60%	67%
0.8 (cautious)	3	0	7	**100%**	30%	46%
0.3 (permissive)	10	7	0	59%	**100%**	**74%**



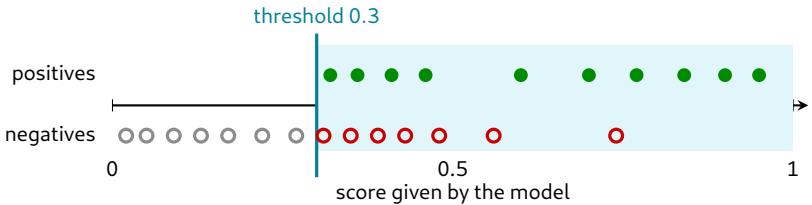
Most classifiers output **one score** (for instance a probability), turned into a class with a threshold (0.5 for instance). Each dot is a test example placed according to its score, positives on top, negatives below; everything right of the threshold is predicted positive. Raising the threshold to 0.8 removes all false alarms (precision 100%) but misses 7 positives (recall 30%). Lowering it to 0.3 catches every positive (recall 100%) with 7 false alarms (precision 59%). The threshold trades precision for recall *without retraining*; all possible thresholds draw the **precision/recall curve**.

The F_1 score summarises each line in one number: here it prefers the permissive threshold (74%), but it weights precision and recall equally, which is not always what the application needs.

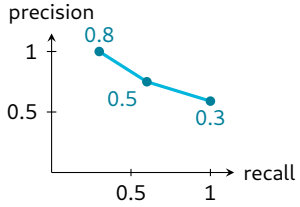
Where to stand is an application decision: a medical screening favours recall (a missed patient is worse than an extra examination), a spam filter favours precision (a lost email is worse than a spam). The threshold is a hyperparameter: choose it on validation data, **never on the test set**. In `sklearn`: `predict_proba` and `precision_recall_curve`.

Precision and recall: a trade-off

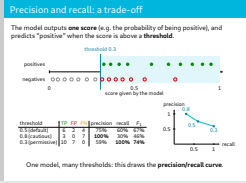
The model outputs **one score** (e.g. the probability of being positive), and predicts "positive" when the score is above a **threshold**.



threshold	*TP*	*FP*	*FN*	precision	recall	F_1
0.5 (default)	6	2	4	75%	60%	67%
0.8 (cautious)	3	0	7	**100%**	30%	46%
0.3 (permissive)	10	7	0	59%	**100%**	**74%**



One model, many thresholds: this draws the **precision/recall curve**.



Most classifiers output **one score** (for instance a probability), turned into a class with a threshold (0.5 for instance). Each dot is a test example placed according to its score, positives on top, negatives below; everything right of the threshold is predicted positive. Raising the threshold to 0.8 removes all false alarms (precision 100%) but misses 7 positives (recall 30%). Lowering it to 0.3 catches every positive (recall 100%) with 7 false alarms (precision 59%). The threshold trades precision for recall *without retraining*; all possible thresholds draw the **precision/recall curve**.

The F_1 score summarises each line in one number: here it prefers the permissive threshold (74%), but it weights precision and recall equally, which is not always what the application needs.

Where to stand is an application decision: a medical screening favours recall (a missed patient is worse than an extra examination), a spam filter favours precision (a lost email is worse than a spam). The threshold is a hyperparameter: choose it on validation data, **never on the test set**. In `sklearn`: `predict_proba` and `precision_recall_curve`.

What about several classes?

One class at a time ("one-vs-rest")

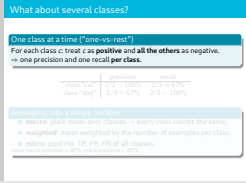
For each class c: treat c as **positive** and **all the others** as negative.
$\Rightarrow$ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes $\rightarrow$ every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the *TP*, *FP*, *FN* of all classes.

Here: macro precision $\approx 83\%$, micro precision $= 80\%$.



Precision and recall are defined for one positive class against the rest, so with C classes we do it C times: the **one-vs-rest** decomposition. The result is a small table, one line per class, usually more informative than any single number since it shows which class the model handles badly.

To obtain one number the per-class values are averaged, and the choice of averaging matters as much as the metric.

Macro gives every class the same weight whatever its size, so a tiny class handled terribly visibly drags the score down (it's generally what you want on imbalanced data). **Weighted** weights each class by its number of examples, so the majority class dominates again. **Micro** pools all the counts; in a single-label problem every error is an *FP* for one class and an *FN* for another.

Reading `classification_report`: one line per class (precision, recall, F1, support), then `macro avg` and `weighted avg`. Read the per-class lines first, and check the support.

What about several classes?

One class at a time ("one-vs-rest")

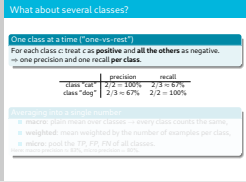
For each class c: treat c as **positive** and **all the others** as negative.
$\Rightarrow$ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes $\rightarrow$ every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the *TP*, *FP*, *FN* of all classes.

Here: macro precision $\approx 83\%$, micro precision $= 80\%$.



Precision and recall are defined for one positive class against the rest, so with C classes we do it C times: the **one-vs-rest** decomposition. The result is a small table, one line per class, usually more informative than any single number since it shows which class the model handles badly.
To obtain one number the per-class values are averaged, and the choice of averaging matters as much as the metric.
Macro gives every class the same weight whatever its size, so a tiny class handled terribly visibly drags the score down (it's generally what you want on imbalanced data). **Weighted** weights each class by its number of examples, so the majority class dominates again. **Micro** pools all the counts; in a single-label problem every error is an *FP* for one class and an *FN* for another.

Reading `classification_report`: one line per class (precision, recall, F1, support), then `macro avg` and `weighted avg`. Read the per-class lines first, and check the support.

What about several classes?

One class at a time ("one-vs-rest")

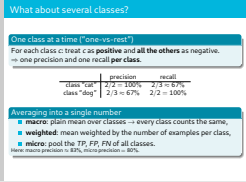
For each class c: treat c as **positive** and **all the others** as negative.
$\Rightarrow$ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes $\rightarrow$ every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the *TP*, *FP*, *FN* of all classes.

Here: macro precision $\approx$ 83%, micro precision $=$ 80%.



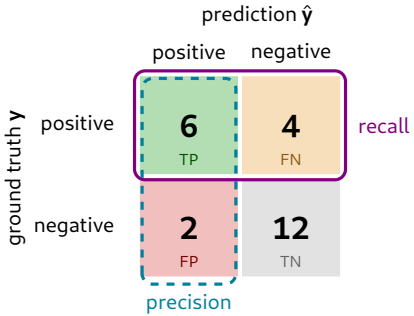
Precision and recall are defined for one positive class against the rest, so with C classes we do it C times: the **one-vs-rest** decomposition. The result is a small table, one line per class, usually more informative than any single number since it shows which class the model handles badly.
To obtain one number the per-class values are averaged, and the choice of averaging matters as much as the metric.
Macro gives every class the same weight whatever its size, so a tiny class handled terribly visibly drags the score down (it's generally what you want on imbalanced data). **Weighted** weights each class by its number of examples, so the majority class dominates again. **Micro** pools all the counts; in a single-label problem every error is an *FP* for one class and an *FN* for another.

Reading `classification_report`: one line per class (precision, recall, F1, support), then `macro avg` and `weighted avg`. Read the per-class lines first, and check the support.

Confusion matrix

Confusion matrix

All the errors at once: **rows** = ground truth **y**, **columns** = prediction $\hat{\mathbf{y}}$ (convention of `sklearn`).

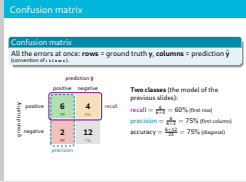


Two classes (the model of the previous slides):

$\text{recall} = \frac{6}{6+4} = 60\%$ (first row)

$\text{precision} = \frac{6}{6+2} = 75\%$ (first column)

$\text{accuracy} = \frac{6+12}{24} = 75\%$ (diagonal)



A confusion matrix is a complete summary of a classifier (actually, every metric above can be recomputed from it). Cell (i, j) counts the test examples of true class i predicted as class j. Some books use the transposed convention: always check the axis titles.

With two classes only, the four cells are the *TP*, *FN*, *FP*, *TN* of the previous slides. Recall reads along the first *row*, precision along the first *column*, accuracy on the *diagonal*.

With several classes, the diagonal holds the correct predictions; the position of each mistake says which classes are confused. Here 12 dogs were taken for cats, but only 6 cats for dogs: the errors are not symmetric, and "dog" is the class to work on.

In the lab, compute it on train **and** test (`confusion_matrix`, `ConfusionMatrixDisplay`): a clean diagonal on train and a messy one on test is the signature of overfitting.

Confusion matrix

Confusion matrix

All the errors at once: **rows** = ground truth **y**, **columns** = prediction $\hat{\mathbf{y}}$ (convention of `sklearn`).

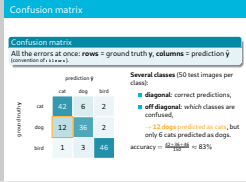
ground truth **y** \ prediction $\hat{\mathbf{y}}$	cat	dog	bird
cat	42	6	2
dog	12	36	2
bird	1	3	46

Several classes (50 test images per class):

- **diagonal**: correct predictions,
- **off diagonal**: *which* classes are confused,

  → **12 dogs** predicted as cats, but only 6 cats predicted as dogs.

accuracy $= \frac{42+36+46}{150} \approx 83\%$



A confusion matrix is a complete summary of a classifier (actually, every metric above can be recomputed from it). Cell (i, j) counts the test examples of true class i predicted as class j. Some books use the transposed convention: always check the axis titles.

With two classes only, the four cells are the *TP*, *FN*, *FP*, *TN* of the previous slides. Recall reads along the first *row*, precision along the first *column*, accuracy on the *diagonal*.

With several classes, the diagonal holds the correct predictions; the position of each mistake says which classes are confused. Here 12 dogs were taken for cats, but only 6 cats for dogs: the errors are not symmetric, and "dog" is the class to work on.

In the lab, compute it on train **and** test (`confusion_matrix`, `ConfusionMatrixDisplay`): a clean diagonal on train and a messy one on test is the signature of overfitting.

Lab Session 2: Supervised Learning

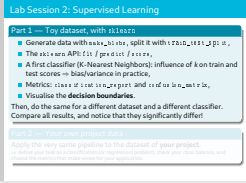
Part 1 — Toy dataset, with `sklearn`

- Generate data with `make_blobs`, split it with `train_test_split`,
- The `sklearn` API: `fit` / `predict` / `score`,
- A first classifier (K-Nearest Neighbors): influence of *k* on train and test scores ⇒ bias/variance in practice,
- Metrics: `classification_report` and `confusion_matrix`,
- Visualise the **decision boundaries**.

Then, do the same for a different dataset and a different classifier. Compare all results, and notice that they significantly differ!

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.
⇒ define your task as a classification (or regression) problem, check your class balance, and choose the metrics that make sense for *your* application.



The lab has the two-part structure of every lab from now on: a controlled toy dataset where everything is visible and you can check that you understood, then your own project data where nothing is clean.

Part 1 is where you build intuitions, because in 2D you can *see* the data and the decision boundaries; do not skip the plots. The scan over *k* in the KNN is the practical counterpart of the bias/variance slide: small *k* follows every point (high variance, perfect training score, poor test score), large *k* over-smooths (high bias). Try to reproduce this behavior!

Part 2 brings back the questions of the beginning of this course: what exactly is your vector **x**, and where does your ground truth **y** come from? Two frequent traps: strongly imbalanced classes (so accuracy is not the right metric), and features with very different scales (some models rely on distances, so scale can mess up the results); look up `StandardScaler`. We will talk about this in the next session.

Lab Session 2: Supervised Learning

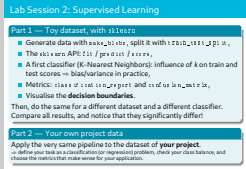
Part 1 — Toy dataset, with `sklearn`

- Generate data with `make_blobs`, split it with `train_test_split`,
- The `sklearn` API: `fit` / `predict` / `score`,
- A first classifier (K-Nearest Neighbors): influence of k on train and test scores $\Rightarrow$ bias/variance in practice,
- Metrics: `classification_report` and `confusion_matrix`,
- Visualise the **decision boundaries**.

Then, do the same for a different dataset and a different classifier. Compare all results, and notice that they significantly differ!

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.
$\Rightarrow$ define your task as a classification (or regression) problem, check your class balance, and choose the metrics that make sense for *your* application.


Lab Session 2: Supervised Learning

Part 1 — Toy dataset, with sklearn

- Generate data with make_blobs, split it with train_test_split,
- The sklearn API: fit / predict / score,
- A first classifier (K-Nearest Neighbors): influence of k on train and test scores ⇒ bias/variance in practice,
- Metrics: classification_report and confusion_matrix,
- Visualise the decision boundaries.

Then, do the same for a different dataset and a different classifier. Compare all results, and notice that they significantly differ!

Part 2 — Your own project data

Apply the very same pipeline to the dataset of your project.
⇒ define your task as a classification (or regression) problem, check your class balance, and choose the metrics that make sense for your application.

The lab has the two-part structure of every lab from now on: a controlled toy dataset where everything is visible and you can check that you understood, then your own project data where nothing is clean.
Part 1 is where you build intuitions, because in 2D you can *see* the data and the decision boundaries; do not skip the plots. The scan over k in the KNN is the practical counterpart of the bias/variance slide: small k follows every point (high variance, perfect training score, poor test score), large k over-smooths (high bias). Try to reproduce this behavior!

Part 2 brings back the questions of the beginning of this course: what exactly is your vector $\mathbf{x}$, and where does your ground truth $\mathbf{y}$ come from? Two frequent traps: strongly imbalanced classes (so accuracy is not the right metric), and features with very different scales (some models rely on distances, so scale can mess up the results); look up `StandardScaler`. We will talk about this in the next session.